Factory Compiler Architecture — Research Report

1. Executive Summary
    - The Problem
    - The Solution: Factory as a Compiler
    - Top 5 Architectural Decisions
    - Key External Analogies
2. Compiler Architecture Patterns
    - 2.1 The Three-Tier Compiler Model
    - 2.2 Intermediate Representations
    - 2.3 Multi-Pass Architecture
    - 2.4 LLVM Pass Manager — The Gold Standard
    - 2.5 MLIR — Multi-Level IR as a Pipeline Architecture Model
    - 2.6 LLVM Pass Dependency and Scheduling Model (Legacy PM)
    - 2.7 GCC Pipeline — Multi-IR Progressive Refinement
    - 2.8 Inter-Pass State Management
    - 2.9 Minimum Viable IR for a Code Generation Pipeline
    - 2.10 Error Recovery: Within Passes vs Between Passes
    - 2.11 Phase Grouping and Data Flow
3. Build System Primitives
    - 3.1 “Build Systems à la Carte” — Theoretical Framework
    - 3.2 Nix: Content-Addressed Store and Hermetic Builds
    - 3.3 Bazel: Action Graph and Content-Addressable Caching
    - 3.4 Shake: Dynamic Dependencies and Early Cutoff
    - 3.5 Build System Primitives for Factory Design
4. CI/CD Pipeline Patterns
    - 4.1 Tekton: Kubernetes-Native Task Isolation
    - 4.2 Argo Workflows: DAG Execution and Retry
    - 4.3 Dagger: Content-Addressed Pipeline Engine
    - 4.4 Stage Retry vs Pipeline Retry
    - 4.5 Andon Cord: “Stop the Line” in CI/CD
5. Agent Orchestration Patterns
    - 5.1 DSPy: Compiling LM Programs
    - 5.2 SWE-bench: The Gold Standard for Agent Output Verification
    - 5.3 METR: Multi-Axis Agent Evaluation
    - 5.4 Agent Behavioral Contracts (ABC) — Formal Output Contracts
    - 5.5 AutoGen: Conversation Patterns and Termination

- 5.6 Agent Sandboxing Patterns

6. State Management Design
- 6.1 Event Sourcing: Append-Only State for Pipelines
- 6.2 Saga Pattern: Distributed Transactions with Compensation
- 6.3 CRDTs: Conflict-Free Concurrent State
- 6.4 Git as Pipeline State Manager
- 6.5 Write-Ahead Logging for Pipeline Consistency

7. Formal Verification Approaches
- 7.1 TLA+ for Pipeline Invariant Specification
- 7.2 Design by Contract for Pipeline Stages
- 7.3 Property-Based Testing for Pipelines

8. Task Specification Schema
- 8.1 Input Grammar Design Patterns
- 8.2 Existing Task Specification Models (Comparison)
- 8.3 Minimum Viable Task Specification Fields
- 8.4 Gherkin-Style Verification Criteria

9. Error Recovery Architecture
- 9.1 Factory Error Taxonomy
- 9.2 Recovery Decision Tree
- 9.3 Compensation Registry

10. Manufacturing Patterns
- 10.1 Toyota Production System: Foundational Principles
- 10.2 Theory of Constraints: Finding the Bottleneck
- 10.3 WIP Limits: Controlling Pipeline Concurrency
- 10.4 Quality Gates vs Quality Built-In
- 10.5 Statistical Process Control for Pipeline Health

11. Existing Implementations Survey
- 11.1 OpenHands (formerly OpenDevin)
- 11.2 Aider
- 11.3 Devin (Cognition)
- 11.4 Cursor 2.0 (Composer)
- 11.5 Cross-Implementation Pattern Analysis

12. SYNTHESIS: Factory Compiler Architecture Spec
- 12.1 Architecture Overview
- 12.2 The Factory Pipeline (Redesigned)
- 12.3 Key Architectural Decisions
- 12.4 Task IR Schema (Final)
- 12.5 Pass Properties and Scheduling
- 12.6 Deep Dive: MLIR Dialect Conversion as Factory Model

12.7 Deep Dive: Nix Store Path Computation and Content-Addressing
12.8 Deep Dive: LangGraph vs Factory Architecture
12.9 Deep Dive: Clash and Merge Conflict Prediction
12.10 Deep Dive: Property-Based Testing for Pipeline Correctness
12.11 Deep Dive: Container vs Worktree Isolation — Factory's Largest Security Gap
12.12 Deep Dive: Merge Queue Architecture
12.13 Deep Dive: Cross-Run Learning (v2 Architecture)
12.14 Deep Dive: Agent Performance Reality Check
12.15 What Factory Gets That Nobody Else Has
12.7 Known Gaps and Future Work
13. Sources

Factory Compiler Architecture — Research Report

> **Status:** Complete — 67 entries across all 10 angles | **Last updated:** 2026-03-16
> **Entries:** 67 | **Coverage:** All 10 research angles + synthesis spec + deep dives across all dimensions

---

1. Executive Summary

Status: Research complete across all 10 angles | **Entries:** 67 | **Last updated:** 2026-03-16

The Problem

The current Factory (orchestrate.sh) is a 9-stage bash script that works when everything goes right but fails catastrophically on reruns, concurrent execution, and agent misbehavior. Root causes: no intermediate representation, no idempotency, shared mutable state (bash variables), no formal output contracts, no compensation for failures.

The Solution: Factory as a Compiler

Factory should be rebuilt as a **compiler-like pipeline with MLIR-style progressive lowering**:

1. **Frontend:** Parse task spec → validate → dedup check (cache hit = skip) → risk classification
2. **Middle-end:** Dependency analysis → conflict prediction → model selection → constitution generation

3. **Backend:** Worktree creation → agent execution (impure, sandboxed, contract-enforced) → verification (SWE-bench model) → security review → merge

Top 5 Architectural Decisions

1. **Progressive lowering** (MLIR): Raw task → structured spec → execution plan → verified code. Not one-shot agent dispatch.
2. **Content-addressed action cache** (Bazel/Nix): `hash(task + commit + model)` → cached diff. Solves idempotency.
3. **Agent Behavioral Contracts** (ABC paper): Formal C=(P,I,G,R) with hard/soft invariants enforced during execution.
4. **Event sourcing + saga compensation**: Every state transition logged, every transform stage has a rollback action.
5. **SWE-bench verification**: FAIL_TO_PASS + PASS_TO_PASS for every task. Binary pass/fail. No partial credit.

Key External Analogies

- **Dagger.io** is the closest existing system (immutable DAG, content-addressed caching, typed modules)
- **Nix fixed-output derivations** handle the impure agent execution problem (run in sandbox, verify output)
- **Erlang rest_for_one** supervision: restart from failed stage, preserve cached analysis
- **Toyota jidoka**: Build quality into execution (ABC invariants), don't just inspect after

---

2. Compiler Architecture Patterns

2.1 The Three-Tier Compiler Model

Modern compilers universally adopt a **frontend → middle-end → backend** architecture (Compiler - Wikipedia, GeeksforGeeks: Grouping of Phases):

Tier	Responsibility	Depends on
Frontend	Lexical analysis, parsing, semantic analysis, IR generation	Source language
Middle-end	Optimization passes on IR	Neither source nor target
Backend	Instruction selection, register allocation, code emission	Target architecture

Key insight for Factory: The middle-end is source-and-target-independent. This is the modularity principle we need — a “task IR” that decouples task specification (frontend) from agent execution (backend). The current pipeline has no middle-end; task specs go directly to agents.

2.2 Intermediate Representations

IRs are the data structures that connect compiler phases. Three major forms exist (GeeksforGeeks: Intermediate Code Generation):

1. **Three-Address Code (TAC):** `x = y op z` — linear, easy to optimize. Represented as quadruples (op, arg1, arg2, result), triples, or indirect triples.
2. **Syntax Trees / DAGs:** Tree-structured IR preserving hierarchical relationships. DAGs share common subexpressions.
3. **Static Single Assignment (SSA):** Each variable assigned exactly once. Used by LLVM, GCC, and most modern optimizers (SSA Form - Wikipedia). Phi functions resolve values at control-flow merge points.

Properties of a good IR: - **Accurate:** Represents source without information loss - **Source-independent:** Not tied to any specific input language - **Target-independent:** Not tied to any specific output platform - **Optimizable:** Amenable to analysis and transformation - **Machine-readable:** Structured enough for automated processing

> **Factory analogy:** The current pipeline has no IR. Task descriptions are freeform markdown (DEBT.md items), and the "compiled" output is whatever the agent happens to produce. We need a structured task IR that captures: authorized file set, expected changes, verification criteria, blast radius, and dependencies. This IR would be the contract between the "frontend" (task classification/risk assessment) and the "backend" (agent execution).

2.3 Multi-Pass Architecture

Compilers process code in **passes** — each pass reads an input representation, transforms it, and produces output (GeeksforGeeks: Single Pass vs Multi-Pass):

- **Single-pass:** All phases in one sequential scan. Fast but limited (can't do forward references).
- **Two-pass:** Frontend (analysis) then backend (synthesis). The classic split.
- **Multi-pass:** 5-50+ passes in modern compilers. Each pass does one well-defined thing.

Modern compilers like LLVM typically run 20-30 passes for `-O2` and 30-50 for `-O3`. Each pass is small, composable, and testable independently.

Key insight for Factory: The current 9-stage pipeline is already multi-pass. But unlike compiler passes, our stages have no formal input/output contracts, no dependency declarations, and no invalidation model. Each stage communicates via bash variables and side effects rather than a typed IR.

2.4 LLVM Pass Manager — The Gold Standard

LLVM's New Pass Manager (NPM) provides the most sophisticated pass pipeline architecture in production use (LLVM New Pass Manager):

Pass Types

- **Analysis passes:** Compute information about code (dominator trees, alias analysis). Read-only — never modify IR.
- **Transform passes:** Modify IR based on analysis results. Must declare what analyses they preserve.

Dependency Model

- Passes don't declare dependencies explicitly — they **request analyses on demand** from an AnalysisManager.
- The AnalysisManager **caches** results and only recomputes when invalidated.
- After a transform pass runs, it returns a `PreservedAnalyses` object declaring what's still valid:
  - `PreservedAnalyses::all()` — nothing was changed
  - `PreservedAnalyses::none()` — conservatively invalidate everything
  - Selective: `preserve<SpecificAnalysis>()` — fine-grained preservation

Hierarchical Pipeline

The IR hierarchy is **Module → (CGSCC →) Function → Loop**. Passes at each level can only access cached, immutable results from outer levels — prevents quadratic blowup and enables future parallelism.

Pipeline Scheduling

- The `PassBuilder` constructs standard pipelines (`-O2` , `-O3`)
- Extension points allow injecting custom passes at specific stages
- Formal grammar validates pipeline structure before execution

> **Factory design implications:** 1. **Separate analysis from transform.** Risk classification, dependency analysis, and blast radius computation should be analysis passes. Worktree creation, agent execution, and merge should be transform passes. 2. **Cache analysis results.** Don't recompute import-graph.json every run — cache it and invalidate when files change. 3. **Declare preservation.** After each stage, declare what's still valid. If the agent modified files outside its authorized set, invalidate the blast radius analysis. 4. **Hierarchical scoping.** Task-level passes should be nested inside project-level passes, just as function passes nest inside module passes.

2.5 MLIR — Multi-Level IR as a Pipeline Architecture Model

MLIR (Multi-Level Intermediate Representation) extends LLVM's single-level IR model to support **multiple abstraction levels simultaneously** (MLIR Introduction - Stephen Diehl, MLIR Paper):

Core Architecture

- **Dialects:** Extensible, domain-specific sets of operations and types. High-level dialects (`tensor`, `linalg`, `gpu`) coexist with low-level ones (`scf`, `memref`, `arith`). New domains add new dialects without modifying the core.
- **Operations:** The basic computational unit — richer than LLVM instructions, carrying dialect namespace and type annotations.
- **Regions:** Group operations, represent control flow boundaries (loops, conditionals). Can take arguments.
- **Blocks:** Linear execution sequences within regions. Can take arguments (unlike LLVM basic blocks).
- **Modules:** Top-level container. Hierarchy: Module → Function → Region → Block → Operation.

Progressive Lowering

Compilation happens through **sequential dialect conversion**: high-level dialects → structured control flow (`scf`) → control flow (`cf`) → LLVM dialect → LLVM IR. Pass ordering matters — `convert-scf-to-cf` must come before `convert-cf-to-llvm`.

Pass Infrastructure

Unlike LLVM's optimization-centric passes, MLIR passes target **dialect conversion** — transforming operations from one abstraction level to another. This is fundamentally a **pipeline of progressive refinement**, not just optimization.

> **Critical Factory insight:** MLIR's progressive lowering model maps directly to the Factory pipeline:
>
> | MLIR Level | Factory Equivalent |
> |---|---|
> | High-level dialect (tensor ops) | Task specification (DEBT.md item) |
> | Structured control flow dialect | Task IR (structured spec with file set, blast radius, contracts) |
> | Low-level dialect (memref, arith) | Agent execution plan (specific Claude CLI invocation + session package) |
> | LLVM IR | Generated code (git diff) |
> | Machine code | Merged commit on main |
>
> Each "lowering" step transforms the representation while **preserving semantic intent**. The task specification "I need to fix the login bug" gets progressively refined into a concrete, verifiable execution plan. This is exactly the missing abstraction in our pipeline — we jump from high-level task description directly to agent execution with no intermediate refinement.

2.6 LLVM Pass Dependency and Scheduling Model (Legacy PM)

The LLVM Legacy Pass Manager provides a well-studied model for pass dependency management (LLVM Writing a Pass):

Pass Type Hierarchy (from least to most constrained)

1. **ImmutablePass** — never runs, provides static config (e.g., target data layout)
2. **ModulePass** — whole-program scope, unpredictable function ordering
3. **CallGraphSCCPass** — bottom-up call graph traversal, must preserve CallGraph
4. **FunctionPass** — per-function, no cross-function state, predictable local behavior
5. **LoopPass / RegionPass** — specialized nested structure processing
6. **MachineFunctionPass** — post-codegen, cannot modify LLVM IR

Dependency Declaration via `getAnalysisUsage`

- `addRequired<Analysis>()` — this pass needs Analysis to run first
- `addRequiredTransitive<Analysis>()` — this pass AND its dependents need Analysis
- `addPreserved<Analysis>()` — this pass doesn't invalidate Analysis
- `setPreservesAll()` — this pass is read-only
- `setPreservesCFG()` — this pass doesn't change control flow

Critical constraint: A pass can only require **analysis** passes, never transform passes. This prevents circular dependencies and makes scheduling always solvable.

Scheduling Optimization

The PassManager pipelines execution — runs all FunctionPasses on function 1, then all on function 2. This improves cache locality. It also caches analysis results and frees them via `releaseMemory()` when no longer needed.

> **Factory design pattern:** Map this to Factory stages: - **ImmutablePass** → Governance files (CLAUDE.md, risk contract) — loaded once, never recomputed - **ModulePass** → Project-wide analysis (import graph, TypeScript build) - **FunctionPass** → Per-task processing (risk classification, worktree creation, agent execution) - **Dependency declaration** → Each stage declares what analyses it

needs and what it preserves - **No transform→transform dependency** → Agent execution (transform) cannot require another transform to run first; it can only require analyses

2.7 GCC Pipeline — Multi-IR Progressive Refinement

GCC uses a distinct three-IR model that provides a clear example of **progressive refinement** through increasingly constrained representations (GCC Compilation Pipeline - DeepWiki, GCC Internals Overview):

IR Progression

```
Source → [Frontend] → GENERIC → [Gimplification] → GIMPLE (High)
  → [Lowering] → GIMPLE (Low) → [SSA Construction] → GIMPLE-SSA
  → [Optimization] → GIMPLE-SSA (optimized) → [RTL Expansion] → RTL
  → [Backend Optimization] → RTL (optimized) → [Code Emission] → Assembly
```

Key IRs and Their Constraints

IR	Constraint	Purpose
GENERIC	Language-independent tree, preserves all constructs	Frontend convergence point — all languages produce same form
GIMPLE	Three-address code: max 3 operands, no nested expressions, explicit temporaries	Enables language-independent optimization
GIMPLE-SSA	Each variable assigned exactly once, PHI nodes at merge points	Enables dataflow analysis, constant propagation, DCE
RTL	Register-based operations, LISP-like s-expressions, target instruction patterns	Machine-level optimization before final emission

GIMPLE Sub-Levels (Progressive Lowering Within One IR)

GIMPLE itself has three sub-forms: **high gimple** (retains control flow, exception handling), **low gimple** (rewrites these into jumps/CFG), and **SSA gimple** (single assignment form). This is progressive lowering *within* a single IR — exactly what MLIR formalizes with dialect conversion.

Pass Property Flags

GCC passes declare properties they require/produce via flags: `PROP_cfg` (control flow graph available), `PROP_ssa` (SSA form active). This is a simpler version of LLVM's `PreservedAnalyses` — binary flags rather than fine-grained analysis tracking.

> **Factory insight: Three-IR Model Applied** GCC's GENERIC→GIMPLE→RTL maps to Factory as: 1. **GENERIC** = Raw task description (DEBT.md markdown, freeform) — preserve all info, no constraints 2. **GIMPLE** = Structured task spec (JSON schema: file set, blast radius, deps, verification criteria) — constrained, machine-readable, no ambiguity 3. **RTL** = Agent execution plan (specific worktree path, model selection, session package contents, timeout) — target-specific, ready to execute
>
> The key insight from GCC's sub-levels: even within the "GIMPLE" (structured task spec) level, there can be progressive refinement — a high-level spec gets risk-classified, then dependency-analyzed, then blast-radius-computed, each step adding constraints while preserving the original intent.

2.8 Inter-Pass State Management

How do compilers actually pass state between passes? Three mechanisms dominate:

1. The IR Itself (Primary State)

The IR is the main state object — passes read it, transform it, and hand it to the next pass. In LLVM, the `Module`, `Function`, and `BasicBlock` objects **are** the state. Passes mutate them in place (transform passes) or read them (analysis passes).

2. Symbol Table (Cross-Phase Persistent State)

The symbol table is the classic cross-phase data structure (GeeksforGeeks: Symbol Table): - **Populated by:** Lexer and parser (name, type, scope) - **Enriched by:** Semantic analysis (type checking, scope resolution) - **Consumed by:** Code generator (variable sizes, addresses), optimizer (type-based optimizations) - **Lifetime:** Entire compilation — survives all passes - **Implementation:** Hash tables in production compilers

Key insight: In multi-pass compilers, symbol tables **must persist** even entries for temporarily inaccessible scopes, because later passes may need them. Single-pass compilers can discard entries on scope exit.

3. Analysis Manager Cache (Computed State)

LLVM's `AnalysisManager` provides lazy-computed, cached analysis results (LLVM AnalysisManager): - **Storage:** `AnalysisResults` map (analysis ID + IR unit → cached result) - **Linear removal:** `AnalysisResultLists` for O(n) invalidation of all results for an IR unit - **Lifecycle:** Computed on demand, cached until invalidated by a transform pass's `PreservedAnalyses` - **Metadata:** LLVM also supports `!annotation` metadata attached directly to IR instructions for lightweight cross-pass communication

4. Persistent/Immutable Data Structures (Functional Approach)

Some compilers use persistent data structures where modifications create new versions while preserving originals (Persistent Data Structures - Wikipedia): - **Structural sharing:** Modified and original versions share unchanged nodes - **Audit trail:** Previous IR versions remain accessible (useful for debugging, rollback) - **Thread safety:** Immutable by construction — safe for concurrent passes

> **Factory state management implications:** 1. **The "IR" is the task manifest** — a JSON/YAML file that accumulates information as it flows through stages. Each stage reads it, enriches it, passes it forward. 2. **Symbol table → Session registry** — maps task IDs to worktree paths, agent PIDs, output files, evidence. Persists across all stages. 3. **Analysis cache → Computed results** — import graph, risk tier, blast

> radius. Computed once, cached, invalidated when underlying files change. 4. **Persistent data structures → Git** — git is already a content-addressed persistent data structure. Each commit is an immutable snapshot. This is a natural fit for Factory state management. 5. **CRITICAL: No bash variables as state.** The current pipeline stores state in bash variables and side effects. This is the equivalent of a compiler storing analysis results in global variables — impossible to cache, invalidate, or audit.

2.9 Minimum Viable IR for a Code Generation Pipeline

Drawing from compiler theory (Q1-Q4) and existing task specification schemas, we can define what a Factory IR needs.

Existing Task Specification Models

SWE-bench defines the gold standard for coding task specs (SWE-bench Datasets): - `instance_id` — unique identifier - `repo`, `base_commit` — exact codebase state - `problem_statement` — natural language description - `patch` — gold solution (for evaluation) - `test_patch` — tests that verify the fix - `FAIL_TO_PASS` — tests that must change from failing to passing - `PASS_TO_PASS` — tests that must continue passing

DSPy signatures define typed input/output contracts for LM modules (DSPy Signatures): - Declarative: `"context: list[str], question: str -> answer: str"` - Type-validated with Python typing system - Composable — signatures chain into pipelines - The compiler optimizes signatures into prompts or finetuning data

agents.json defines API interaction contracts for AI agents (agents.json): - Built on OpenAPI standard - **Flows:** Series of API calls describing an outcome - **Links:** How two actions stitch together - Discovery via `/.well-known/agents.json`

Tekton Tasks define CI/CD task specs (Tekton Pipeline API): - Typed parameters (string, array) - Workspace declarations (volumes required) - Results (outputs consumable by next tasks) - Steps (sequential container executions)

Synthesis: Factory Task IR Design

Drawing from all sources, a minimum viable Factory IR needs these fields:

```yaml
# Level 1: Task Identity (GENERIC equivalent)
task_id: string          # content-addressed hash of spec
source: string           # DEBT.md item reference or issue URL
description: string      # natural language intent
created_at: timestamp
idempotency_key: string  # hash of (source + base_commit) — prevents duplicate runs

# Level 2: Scope Contract (GIMPLE equivalent — constrained, machine-readable)
base_commit: string      # exact git state this task targets
authorized_files:        # explicit allowlist — agent CANNOT modify other files
  - path: string
    operation: create | modify | delete
blast_radius:            # files that could break if authorized_files change
  - path: string
    risk: high | medium | low
dependencies:            # other task_ids that must complete first
  - task_id: string
    relationship: blocks | informs
risk_tier: critical | high | medium | low

# Level 3: Verification Contract (output typing)
preconditions:           # must be true BEFORE agent runs
  - type: typecheck | test | lint | custom
    command: string
    expected: pass
postconditions:          # must be true AFTER agent runs (SWE-bench model)
  - type: test
    tests_must_pass: string[]     # PASS_TO_PASS equivalent
    tests_must_fix: string[]      # FAIL_TO_PASS equivalent
  - type: typecheck
    command: "pnpm tsc --noEmit"
  - type: diff_constraint
    max_files_changed: number
    forbidden_patterns: string[]  # e.g., no changes to CLAUDE.md

# Level 4: Execution Plan (RTL equivalent — target-specific)
model: string            # claude-sonnet-4-20250514, opus, etc.
worktree_path: string    # isolated execution environment
session_package:         # files to include in agent context
  - path: string
    purpose: governance | context | reference
timeout_seconds: number
max_retries: number
constitution_path: string  # CLAUDE.md for this specific task
```

Key design properties (from compiler IR theory): 1. **Content-addressed:** `task_id` is a hash of the spec, enabling deduplication and caching 2. **Progressive refinement:** Levels 1-4 are populated by successive analysis passes, not all at once 3. **Typed contracts:** Every field has a type — no freeform markdown 4. **Verification built-in:** Pre/postconditions are part of the IR, not a separate system 5. **Immutable once computed:** Like SSA — each field is assigned exactly once 6. **Idempotency key:** Prevents duplicate execution (the #1 current bug)

2.10 Error Recovery: Within Passes vs Between Passes

Error recovery is fundamentally different depending on where the error occurs — within a single pass/stage or between stages.

Within-Pass Error Recovery (Compiler Strategies)

Four classical strategies from compiler theory (GeeksforGeeks: Error Recovery):

Strategy	How it works	Applicable to Factory
Panic mode	Discard input until a synchronizing token (e.g., semicolon) is found	Agent produces invalid output → skip to next natural boundary (next file, next function)
Phrase-level	Local correction — replace comma with semicolon, insert missing token	Agent makes a small mistake → patch it locally (e.g., fix import statement)
Error productions	Augment grammar with rules for known errors	Pre-define common agent mistakes and handle them automatically
Global correction	Find minimum-edit parse tree	Too expensive for real-time — theoretical interest only

Key insight: Panic mode is the simplest and most robust. In Factory terms, if an agent produces garbage for one file, skip that file and try the next. Don't try to fix the agent's output — just mark it as failed and continue.

Between-Pass Error Recovery (Supervision & Resilience)

Erlang/OTP Supervision Trees (Erlang Supervisor Docs):

Strategy	Behavior	Factory Mapping
one_for_one	Restart only the failed child	Retry just the failed stage
one_for_all	Restart all children	Re-run the entire pipeline from scratch
rest_for_one	Restart the failed child and all children started after it	Re-run from the failed stage onward
Restart intensity	If >MaxR restarts in MaxT seconds, supervisor dies	Circuit breaker — stop retrying after N failures

Why restarting works: Bugs are often context-dependent. Rolling back to a known stable state (clean worktree, fresh agent session) often makes the bug disappear. This is directly applicable to Factory — agent misbehavior is often non-deterministic.

Resilience Patterns for Stage Failures (Microservices Resilience Patterns):

- **Circuit breaker:** If agent execution fails 3 times → stop trying, mark task as blocked, alert human. Three states: closed (normal), open (blocked), half-open (test with simple task).
- **Bulkhead:** Isolate each task in its own worktree/process. One task's failure cannot affect another's. The current Factory already does this with git worktrees — this is correct.
- **Retry with jitter:** Don't retry immediately — add random delay to avoid retry storms when multiple tasks fail simultaneously.
- **Compensation:** If a stage partially succeeds (agent committed to worktree but verification fails), compensate by deleting the worktree branch rather than leaving state dangling.

> **Factory error recovery architecture:** 1. **Within a stage:** Use panic mode. If agent output is partially valid, take the valid parts and mark the rest as failed. Don't try to fix agent output. 2. **Between stages:** Use rest_for_one. If verification fails, restart from agent execution (stage 6) onward, not from the beginning. This preserves the expensive analysis work. 3. **Pipeline level:** Circuit breaker. If a task fails 3 times end-to-end, stop and escalate. Don't retry forever. 4. **Compensation:** Every transform

> stage must have a rollback action. Agent execution → delete worktree branch. Merge → revert commit. Security review → no compensation needed (read-only). 5. **CRITICAL: rest_for_one, not one_for_all.** The current pipeline either succeeds or fails entirely (one_for_all). We need the ability to restart from a failed stage while keeping prior analysis results cached.

2.11 Phase Grouping and Data Flow

Compiler phases communicate through well-defined data structures (GeeksforGeeks: Grouping of Phases):

From → To	Data Structure
Lexer → Parser	Token stream
Parser → Semantic Analysis	Parse tree / AST
Semantic Analysis → IR Gen	Annotated AST + symbol table
IR Gen → Optimizer	IR (three-address code / SSA)
Optimizer → Code Gen	Optimized IR

Each boundary has a **typed contract**. The parser doesn't produce arbitrary output — it produces an AST conforming to the language grammar. The optimizer doesn't produce arbitrary output — it produces valid IR with the same semantics.

> **Factory gap:** Our stages have no typed contracts. Stage 6 (agent execution) can produce literally anything — wrong files, commits to main, modified governance files. The verification stage (7) only checks TypeScript types, not semantic correctness or contract compliance. We need typed stage boundaries.

---

3. Build System Primitives

3.1 “Build Systems à la Carte” — Theoretical Framework

The landmark paper by Mokhov, Mitchell, and Peyton Jones (Build Systems à la Carte) provides a formal framework for characterizing build systems along two orthogonal axes:

Scheduler (when do tasks execute?)

Type	Behavior	Example
Topological	Execute in dependency order, never revisit	Make, Nix
Restarting	May restart dependents after rebuilding	Bazel
Suspending	Pause and resume tasks based on dynamic deps	Shake

Rebuilder (what gets rebuilt?)

Type	Behavior	Example
Dirty bit	Rebuild if any dependency changed	Make
Verifying traces	Check if actual inputs match recorded values	Shake
Constructive traces	Record and verify both inputs AND outputs	—
Deep constructive traces	Extend constructive traces recursively	Bazel, Nix

Key Properties

- **Minimality:** Only rebuild what’s necessary given changes
- **Early cutoff:** If a task rebuilds but produces the same output, don’t rebuild dependents (Shake excels here)
- **Correctness:** Output accurately reflects all inputs

> **Factory mapping:**

Property	Current Factory	Correct Factory
Scheduler	None (sequential bash script)	**Suspending** — tasks may need to pause waiting for dependencies
Rebuilder	None (always re-runs everything)	**Verifying traces** — check if inputs changed since last run
Minimality	No — re-runs all 9 stages every time	Yes — skip stages whose inputs haven't changed
Early cutoff	No	Yes — if risk classification produces same result, skip re-execution
Idempotency	No — duplicate commits	Yes — content-addressed outputs prevent duplicates

3.2 Nix: Content-Addressed Store and Hermetic Builds

Nix provides the strongest reproducibility model of any build system (Nix Hermeticity):

Core Principles

- **Content-addressed store:** Build outputs are stored by hash of their contents. Identical inputs → identical hash → cached.
- **Hermetic sandbox:** Builds are isolated from the host — no network access, no filesystem access outside declared inputs. This ensures the same inputs always produce the same outputs.
- **Derivation as spec:** A Nix derivation is a complete, declarative specification of a build: inputs, builder, environment, output paths. This is the Nix equivalent of a "task IR."

Handling Impurity

Nix handles inherently impure operations (network access, timestamps) through **fixed-output derivations** (Nix Content-Addressing): - The output hash is declared in advance - The builder gets network access (impure) - But the output is verified against the declared hash - If the hash doesn't match, the build fails

This is "carefully controlled impurity" — the builder can do anything, but the output is verified.

> **Factory insight:** Agent execution is inherently impure (LLM inference is non-deterministic). Like Nix fixed-output derivations, we should: 1. Let the agent do its thing (impure execution in a sandbox) 2. Verify the output against the declared contract (postconditions) 3. If verification fails, the "build" fails — don't accept the output
>
> The sandbox (git worktree) provides filesystem isolation. Network isolation isn't needed since agents need API access. But we DO need to verify outputs against contracts, which the current pipeline only partially does (TypeScript typecheck only).

3.3 Bazel: Action Graph and Content-Addressable Caching

Bazel provides the most mature remote caching model (Bazel Remote Caching):

Two-Phase Build Model

1. **Analysis phase:** Build rules produce an **action graph** — a DAG of all actions needed
2. **Execution phase:** Traverse action graph, check cache, execute only uncached actions

Action → Digest → Cache → Output

Each action is a “spawn” containing all information needed to execute: - Input files and their digests - Command to run - Environment variables - Platform properties

The **action cache (AC)** maps action digests to output digests. The **content-addressable store (CAS)** maps output digests to actual files.

Cache lookup flow: 1. Compute action digest from inputs + command 2. Check local AC → if hit, check if outputs exist locally 3. Check remote AC → if hit, download outputs from remote CAS 4. If miss, execute locally, upload outputs to CAS, update AC

> **Factory caching model:**
>
> ```
> Action = hash(task_spec + base_commit + model_version + constitution)
> Cache lookup: AC[action_digest] → output_digest
> CAS[output_digest] → verified_diff
> ```
>
> If the same task with the same base commit was already successfully executed and verified, **skip re-execution entirely**. This solves the idempotency problem — the first run caches its output, subsequent runs find the cache hit.

3.4 Shake: Dynamic Dependencies and Early Cutoff

Shake adds a critical capability missing from Make and Bazel: **dynamic dependencies** (Shake Build System):

- Dependencies are discovered during execution, not declared upfront
- **Verifying traces:** After a build, Shake records what each rule consumed. On rebuild, it checks if those inputs actually changed.
- **Early cutoff:** If a rule rebuilds but produces the same output (via `Eq` comparison), dependents are NOT rebuilt. This is huge for avoiding cascading rebuilds.

> **Factory early cutoff example:** Stage 3 (risk classification) takes the task spec and produces a risk tier. If the task spec changes slightly (e.g., description updated) but the risk tier remains the same → don't re-run stages 4-9. Early cutoff propagation.

3.5 Build System Primitives for Factory Design

Synthesizing across Nix, Bazel, and Shake:

Primitive	What it does	Factory Application
Content-addressing	Dedup and cache by hash of content	Task IR → content hash = idempotency key
Hermetic sandbox	Isolate build from host	Git worktree + restricted agent permissions
Derivation/action spec	Declarative build description	Task IR (our Level 1-4 spec from §2.9)
Action cache	Map (inputs → outputs)	Cache verified diffs by task+commit hash
Early cutoff	Don't rebuild if output unchanged	Skip downstream stages if analysis produces same result
Dynamic dependencies	Discover deps during execution	Agent discovers import dependencies at runtime
Fixed-output derivation	Impure execution + output verification	Agent execution (impure) + postcondition checking
Verifying traces	Record consumed inputs, verify on rebuild	Record what files the agent actually read/wrote

---

4. CI/CD Pipeline Patterns

4.1 Tekton: Kubernetes-Native Task Isolation

Tekton provides the most explicit typed-contract model for CI/CD tasks (Tekton Pipelines):

Task Isolation Model

- Each Task runs as its own **Kubernetes Pod** — full process and filesystem isolation

- Steps within a Task run as sequential containers in the same Pod
- **Workspaces** provide shared volumes for data passing between Tasks
- WorkspaceDeclaration allows Steps to declare isolated access to specific workspace volumes

Typed Contracts

- **Parameters:** Typed as `string` or `array`, passed from Pipeline to Task
- **Results:** String outputs emitted by Tasks, consumable by downstream Tasks
- **Artifacts:** Input/output artifact declarations with type metadata

> **Factory mapping:** Tekton's model is close to what Factory needs: - Each Factory stage = Tekton Task (isolated execution environment) - Task IR fields = Tekton Parameters + Results - Git worktree = Tekton Workspace (shared volume for data passing) - The limitation: Tekton Results are strings only — not structured objects. Factory needs richer typed contracts.

4.2 Argo Workflows: DAG Execution and Retry

Argo Workflows provides the most mature DAG execution engine (Argo Workflows):

DAG Execution

- Tasks define dependency graph — Argo executes in parallel where possible
- Templates can be DAG or step-based, and can nest (DAG templates calling DAG templates)
- Artifact passing between tasks using S3/GCS/Minio backends
- Template references: `{{tasks.generate-artifact.outputs.artifacts.hello-art}}`

Retry Policies

- Per-task retry with configurable limits and backoff intervals

- Different strategies for different failure types (transient vs permanent)

> **Factory insight:** Argo's DAG model would let us express task dependencies properly. Currently, Factory stages are strictly sequential — but some could run in parallel (e.g., security review and test verification could run simultaneously after agent execution).

4.3 Dagger: Content-Addressed Pipeline Engine

Dagger provides the most architecturally sophisticated pipeline model, and it's the closest existing system to what Factory needs (Dagger Architecture):

Immutable, Content-Addressed DAG

- **Every operation produces new immutable state** — no mutation
- Operations are content-addressed via `call.ID` — hash of receiver chain + operation name + sorted arguments
- **Automatic deduplication:** Identical operations with identical inputs never re-execute
- **Lazy evaluation:** Pipeline constructs a DAG before execution — only executed paths run
- **Multi-layer caching:** Session cache → local disk (BuildKit snapshots) → cloud remote cache

Architecture Layers

1. User Layer (CLI, SDKs in Go/Python/TypeScript)
2. API Layer (GraphQL via dagql.Server)
3. Module System (extensible functions with typed args/returns)
4. Core API Objects (Container, Directory, File, Service, Secret, GitRepository — all immutable)
5. Engine Layer (custom BuildKit)
6. Infrastructure (LLB Solver, caching)

Module System

- Modules define typed functions with cache policies: `CachePerSession`, `CachePerCall`, `NeverCache`
- Cross-language type safety via `TypeDef` system
- Schema merging — modules integrate seamlessly with core API

Dagger is the strongest existing analog for Factory. Key parallels:

Dagger	Factory
Container (immutable, content-addressed)	Task IR (immutable, content-addressed)
BuildKit LLB	Pipeline stage execution
Content-addressed caching	Action cache (skip re-execution)
Module functions	Pipeline stages as composable, typed functions
Lazy DAG evaluation	Build action graph before execution
`NeverCache` policy	Agent execution (non-deterministic, must always run)

Critical difference: Dagger executes deterministic operations (compile, test, deploy). Factory executes a non-deterministic agent. Dagger's caching assumes identical inputs → identical outputs. For Factory, identical inputs → *verified* outputs (Nix fixed-output derivation model).

4.4 Stage Retry vs Pipeline Retry

CI/CD platforms handle failure recovery at different granularities (AWS CodePipeline, Harness):

Level	What restarts	When appropriate
Step retry	Single step within a stage	Transient failures (network timeout, API rate limit)
Stage retry (failed actions only)	Only the failed actions in a stage	Partial stage failure where other actions succeeded
Stage retry (all actions)	Entire stage from scratch	Stage state is corrupted, need clean restart
Pipeline retry (from failed stage)	Failed stage + all downstream stages	Analysis results still valid, execution failed
Full pipeline retry	Everything from scratch	Fundamental input has changed

> **Factory retry strategy:** - Agent execution failure → **stage retry (all actions)** — fresh agent session in clean worktree - Verification failure → **step retry** — re-run just the failing check (maybe TypeScript fixed itself after a second compile) - Merge conflict → **pipeline retry from stage** — re-run from agent execution with updated base commit - Governance file changed → **full pipeline retry** — constitution has changed, everything must re-evaluate

4.5 Andon Cord: "Stop the Line" in CI/CD

The Toyota Andon Cord pattern (DevOps.com, IT Revolution):

- Any worker can pull the cord to **stop the production line** when they find a defect
- The key insight: stopping production when a defect is found is cheaper than letting the defect flow downstream
- In CI/CD: compile errors, security scan failures, test failures → automatic cord pull
- **Everyone must see the failure** — visibility is critical

> **Factory andon:** - Agent modifies governance files → **STOP** — this is the highest-severity defect - Agent commits to main branch → **STOP** — sandbox violation - Security review finds vulnerability → **STOP** — don't merge - TypeScript typecheck fails → stage failure, retry - Current Factory has no andon cord — failures are silent. Need a notification system that makes failures visible and blocks the pipeline.

---

5. Agent Orchestration Patterns

5.1 DSPy: Compiling LM Programs

DSPy treats LM calls as compilable, optimizable modules rather than manual prompts (DSPy, MIPROv2):

Compilation Model

- **Signatures** declare typed I/O: `"context: list[str], question: str -> answer: str"`
- **Modules** wrap LM calls with strategy (ChainOfThought, ReAct, etc.)
- **Optimizers** (MIPROv2) compile programs by:
  1. **Bootstrapping:** Run program many times, collect traces, keep high-scoring ones
  2. **Instruction proposal:** Generate candidate instructions from dataset summaries + code + traces
  3. **Bayesian optimization:** Search for best instruction + few-shot example combinations

> **Factory insight: DSPy's compilation model suggests Factory should optimize its agent prompts.** The current constitution (CLAUDE.md) is hand-written. DSPy-style optimization could: bootstrap from successful task completions, propose better instructions per task type, and search for optimal constitution configurations. This is a v2+ feature, not v1.

5.2 SWE-bench: The Gold Standard for Agent Output Verification

SWE-bench provides the most rigorous agent evaluation model for coding tasks (SWE-bench Evaluation, SWE-bench GitHub):

Evaluation Architecture

1. **Docker containerization:** Each task runs in a Docker image capturing exact repo snapshot, pinned deps, correct build toolchain
2. **Patch application:** Agent's output (git diff) is applied to the base commit
3. **Test execution:** Run the full test suite in the container
4. **Dual invariant checking:**
   - `FAIL_TO_PASS`: Tests that were failing MUST now pass (the fix works)
   - `PASS_TO_PASS`: Tests that were passing MUST still pass (no regressions)

Key Design Properties

- **Environment isolation:** Docker eliminates environment drift
- **Reproducibility:** Same image + same patch → same result
- **Binary scoring:** Either the patch passes both invariants or it doesn't
- **No partial credit:** A patch that fixes the bug but breaks other tests fails

> **Factory MUST adopt the SWE-bench verification model:** 1. Define `FAIL_TO_PASS` and `PASS_TO_PASS` for every task in the task IR 2. Run verification in an isolated environment (Docker or clean worktree) 3. Binary pass/fail — no partial acceptance 4. The current Factory only checks TypeScript typecheck — this is equivalent to checking that the code compiles but never running tests. Fundamentally insufficient.

5.3 METR: Multi-Axis Agent Evaluation

METR evaluates agent output on multiple quality axes beyond just functional correctness (METR):

- **Functionally correct code** that cannot be used as-is due to quality issues

- Evaluates: missing core functionality, inadequate test coverage, missing documentation, code quality
- **Key finding:** Automatic scoring overestimates agent performance — agents often produce code that passes tests but has quality problems

> **Factory implication:** Verification must go beyond tests. Security review (current stage 8) is correct. But we should also check: does the diff follow project conventions? Are there obvious code smells? Is the change minimal (no unnecessary modifications)?

5.4 Agent Behavioral Contracts (ABC) — Formal Output Contracts

The ABC framework (Agent Behavioral Contracts) provides the formal specification model Factory needs:

Contract Structure: C = (P, I, G, R)

Component	What it specifies	Factory Application
Preconditions (P)	State requirements before execution	Baseline typecheck passes, deps installed, worktree clean
Invariants (I)	Constraints that must hold during execution	Hard: governance files unmodified, no commits to main. Soft: stay within authorized file set
Governance (G)	Action-level operational boundaries	Hard: spending cap (model cost), no forbidden tool invocations. Soft: timeout warnings
Recovery (R)	Corrective actions on soft violation	Re-prompt agent with constraints, revert unauthorized changes

Hard vs Soft Constraints

- **Hard invariants:** Safety-critical, violation = immediate halt. E.g., “governance files never modified”
- **Soft invariants:** Recoverable, trigger recovery within k steps. E.g., “agent touched files outside authorized set — revert and re-prompt”

Probabilistic Satisfaction: (p, δ, k)

- **p:** Probability threshold ($p \geq 0.99$ for critical pipelines)
- **δ:** Tolerable soft violation rate
- **k:** Recovery window in steps

ContractSpec (YAML DSL)

ABC includes a YAML-based specification language with expression predicates, hard/soft markers, and recovery chains. This maps directly to Factory's task IR verification contract.

Runtime Enforcement: AgentAssert

Per-turn enforcement loop: evaluate predicates → detect violations → execute recovery → monitor drift. Sub-10ms overhead per action.

> **This is the missing piece in Factory.** The current system has no formal contract enforcement. The agent can do anything and we only check TypeScript types after the fact. ABC provides: 1. **Precondition checking** before agent runs (not after) 2. **Invariant monitoring** during agent execution (not after) 3. **Hard/soft distinction** — some violations are fatal, others are recoverable 4. **Recovery protocol** — don't just fail, try to recover 5. **Formal specification** — contracts are YAML, not English

5.5 AutoGen: Conversation Patterns and Termination

AutoGen provides patterns for multi-agent orchestration (AutoGen, Termination):

Orchestration Patterns

- **RoundRobinGroupChat:** Agents take turns in fixed order
- **SelectorGroupChat:** Dynamic speaker selection based on context
- **HandoffTermination:** Agent hands off to specific target (human-in-the-loop)
- **Custom orchestrators:** Application-specific routing logic

Termination Conditions

- **MaxMessageTermination:** Stop after N messages (prevent infinite loops)
- **StopMessageTermination:** Agent produces explicit “DONE” token
- **HandoffTermination:** Pause for human input
- **ExternalTermination:** Programmatic stop from outside (UI “Stop” button)
- **SourceMatchTermination:** Stop when specific agent responds

> **Factory termination:** Currently no termination condition except timeout. Should use: - StopMessage: Agent signals completion - MaxMessage: Hard cap on agent turns to prevent runaway - External: Human can kill a stuck task - **Missing in AutoGen but needed:** Contract violation termination — stop when hard invariant violated

5.6 Agent Sandboxing Patterns

Current best practices for agent sandboxing (NVIDIA Sandboxing Guide, OpenAI Codex Sandboxing):

Mandatory Controls

- **Filesystem isolation:** Agent can only access designated directories
- **Network controls:** Block egress to prevent data exfiltration
- **Resource limits:** CPU and memory caps
- **No persistence:** Agent cannot modify files outside workspace

Formal Analysis (SkillFortify)

- **Static analysis:** Catch capability violations before execution (SkillFortify)
- **Runtime sandboxing:** Safety net for dynamically constructed operations
- **Separation of concerns:** Static catches most violations; runtime catches the rest

> **Factory sandboxing assessment:** - Git worktree: ✅ Filesystem isolation (correct) - `--permission-mode bypassPermissions`: ❌ No permission restrictions on agent - Network: ❌ No network isolation (agent has full API access) - Resource limits: ❌ No CPU/memory caps - Static analysis: ❌ No pre-execution analysis of agent plan -

> The `bypassPermissions` flag is a design decision (agent needs to write files) but means the agent can do ANYTHING within the worktree. Need post-hoc verification to compensate.

---

6. State Management Design

6.1 Event Sourcing: Append-Only State for Pipelines

Event sourcing stores state as an **append-only log of events** rather than mutable current state (Solace Event-Driven Patterns):

Core Properties

- **Immutable log:** Events are facts that happened — never modified or deleted
- **State derived from replay:** Current state = fold(initial_state, all_events)
- **Complete audit trail:** Every change is recorded with timestamp, actor, and context
- **Time travel:** Can reconstruct state at any point by replaying events up to that timestamp

Factory Event Sourcing Model

The Factory pipeline naturally produces a stream of events:

```
TaskCreated { task_id, spec, timestamp }
RiskClassified { task_id, tier: "high", blast_radius: [...] }
WorktreeCreated { task_id, path: "/tmp/worktree-abc123" }
AgentStarted { task_id, model: "claude-sonnet-4-20250514", session_id }
AgentCompleted { task_id, diff_hash, files_changed: [...] }
VerificationPassed { task_id, typecheck: "pass", tests: "pass" }
SecurityReviewPassed { task_id, reviewer_model, verdict: "safe" }
MergeCompleted { task_id, commit_sha, target_branch: "main" }
```

> **Key insight:** The current Factory stores state in bash variables (`$TASK_ID` , `$RISK_TIER` , `$WORKTREE_PATH`). These disappear when the process exits. Event sourcing persists every state transition to a log file (or git commits). Benefits: 1. **Post-mortem debugging:** Can replay exactly what happened in any run 2. **Idempotency:**

> Check if `TaskCreated` event already exists → skip 3. **Recovery:** On restart, replay events to reconstruct state, continue from last event 4. **Metrics:** Count events by type to measure pipeline health over time

6.2 Saga Pattern: Distributed Transactions with Compensation

The Saga pattern (Garcia-Molina & Salem, 1987) manages long-running transactions across multiple services (Microservices.io Saga Pattern, Temporal Saga Guide):

Core Architecture

A saga is a sequence of **local transactions**, each with a **compensating transaction** for rollback:

```
T1 → T2 → T3 → T4 (success path)
T1 → T2 → T3 ✗ → C2 → C1 (failure at T3, compensate T2 and T1)
```

Two Coordination Approaches

Approach	How it works	Trade-offs
Orchestration	Central coordinator directs participants	Easier to understand, single point of failure
Choreography	Services react to each other's events	Decentralized, harder to debug

Compensating Transactions

The developer **must design** compensating transactions — no automatic rollback. Each step needs an explicit undo:

Forward Transaction	Compensating Transaction
Create worktree	Delete worktree branch
Agent writes code	`git checkout -- .` in worktree
Merge to main	`git revert <commit>`
Update DEBT.md	Remove added entry

Factory saga model (orchestration): The Factory pipeline IS a saga — a sequence of local transactions (stages) with compensating transactions:

```
LoadHarness → Baseline → Classify → SelectModel → CreateWorktree → ExecuteAgent →
Verify → SecurityReview → Merge
                                                                  ↑
                                                        failure here triggers:
                                                  C-Verify: mark task as failed
                                                  C-Worktree: delete worktree branch
                                                  (analysis results preserved — not compensated)
```

CRITICAL: The current Factory has NO compensating transactions. When a stage fails, partial state is left dangling: orphaned worktree branches, half-written progress.md entries, stale lock files.

6.3 CRDTs: Conflict-Free Concurrent State

CRDTs (Conflict-free Replicated Data Types) enable concurrent updates that **always merge without conflicts** (CRDT.tech, Wikipedia CRDT):

Core Property

All operations are **commutative** — order doesn't matter. Any replica can apply updates independently, and all replicas eventually converge to the same state.

Two Types

Type	How it works	Trade-off
State-based (CvRDT)	Merge entire states via lattice join	Simple but expensive to transmit
Operation-based (CmRDT)	Transmit individual operations	Efficient but requires causal delivery

Practical CRDT Types

- **G-Counter:** Grow-only counter — each node has its own counter, merge = take max per node
- **PN-Counter:** Grow + shrink — two G-Counters (positive and negative)

- **G-Set:** Grow-only set — merge = union
- **OR-Set:** Observed-remove set — add wins over concurrent remove
- **LWW-Register:** Last-writer-wins register — timestamp-based conflict resolution

Factory Applications

- **Progress tracking (G-Set):** Each completed task ID is added to a grow-only set. Concurrent pipeline runs can both add task IDs. Merge = union. No duplicates, no conflicts.
- **DEBT.md dedup (OR-Set):** DEBT items as an OR-Set. Adding an item adds it to the set. Completing an item removes it. Concurrent add+remove → add wins (item stays in set). This prevents the current bug where DEBT.md gets duplicate entries.
- **Pipeline metrics (G-Counter):** Count successful/failed runs per task type. Each pipeline instance increments its own counter. Merge = sum.

> **Factory concurrency model:** Current Factory fails on concurrent execution because it uses **mutable shared state** (DEBT.md, progress.md). CRDTs provide conflict-free alternatives: - Replace DEBT.md append-only with OR-Set (add/remove without conflicts) - Replace progress.md with G-Set of completed task IDs - Replace global counters with G-Counters **Practical note:** We don't need a full CRDT library. Git itself provides eventual consistency through three-way merge. The key insight is: structure shared state so that concurrent modifications are always mergeable (set operations, not overwrites).

6.4 Git as Pipeline State Manager

Git already provides several state management primitives (Git Worktrees, Git Concurrency):

Content-Addressed Storage

- Every git object (blob, tree, commit) is stored by its SHA hash
- Identical content → identical hash → automatic dedup
- This IS a content-addressable store (CAS)

Concurrency Control

- **Lock files:** Git uses `*.lock` files for pessimistic locking on index operations
- **Three-way merge:** Concurrent changes to different files merge automatically
- **Merge conflicts:** Concurrent changes to same lines require resolution
- **Worktree isolation:** Each worktree has its own HEAD, index, and working tree — operations in one worktree don't affect others

Optimistic Concurrency via Compare-and-Swap

Git merge is essentially optimistic concurrency control: 1. Read current state (base commit) 2. Make changes independently (in worktree) 3. Attempt merge (compare-and-swap) 4. If conflict → resolve or abort (no corruption)

> **Factory should use git as its primary state store:** 1. **Task manifests** as JSON files committed to a `factory/` directory 2. **Stage outputs** as files in the task manifest directory 3. **Merge queue** as git branches: `factory/task-{id}/stage-{n}` 4. **Completion** as merge to main (already the current model) 5. **Concurrent isolation** via worktrees (already the current model) 6. **Idempotency check** via commit history: search for task hash in existing commits

6.5 Write-Ahead Logging for Pipeline Consistency

Database WAL ensures consistency by logging intended changes before applying them:

WAL Concept	Factory Equivalent
Log intent before execution	Write task manifest before starting pipeline
Apply changes	Execute pipeline stages
Checkpoint	Commit stage results to git
Replay on crash	On restart, read task manifest, check which stages completed, resume from last checkpoint

> **Factory WAL model:** Before each transform stage, write a "intent" entry: `{ stage: "agent_execution", status: "starting", timestamp }`. After completion, update to `{ stage: "agent_execution", status: "completed", output_hash }`. On crash recovery, find the last completed stage and restart from the next one. This is the **rest_for_one** supervision strategy backed by persistent state.

---

7. Formal Verification Approaches

7.1 TLA+ for Pipeline Invariant Specification

TLA+ is a formal specification language for concurrent systems (TLA+ - Lamport). Key concepts: invariants (state predicates true in ALL reachable states), actions (state transitions), temporal formulas (safety/liveness over sequences), TLC model checker (exhaustive finite-state exploration).

Factory Pipeline Invariants (TLA+ style):

Safety (must ALWAYS hold): - `GovernanceInvariant`: governance_files_hash(worktree) = governance_files_hash(main) - `MainMonotonicity`: main branch only advances, never regresses - `NoDuplicateTasks`: no two completed tasks share an idempotency key - `WorktreeIsolation`: each active task has unique worktree path - `MergeVerified`: nothing merges without verification AND security review

Liveness (must EVENTUALLY hold): - `TaskCompletion`: every task reaches merged/failed/blocked - `CompensationComplete`: every failed transform has compensation executed

7.2 Design by Contract for Pipeline Stages

Bertrand Meyer's DbC (Design by Contract) maps to stage contracts:

Stage	Precondition	Postcondition	Invariant
Risk Classification	Valid task spec JSON	risk_tier populated, blast_radius computed	Spec not modified
Worktree Creation	base_commit exists, no existing worktree	Worktree exists, clean, deps installed	No main changes
Agent Execution	Worktree clean, constitution materialized	Agent exited, diff exists	Governance unchanged
Verification	Diff exists	Tests pass, typecheck passes	Worktree not modified
Security Review	Diff exists, verified	Verdict: safe/unsafe	Worktree not modified
Merge	Verified + secure, main not diverged	Commit on main with evidence	Main advances monotonically

7.3 Property-Based Testing for Pipelines

Property-based testing verifies universal properties: (1) Idempotency — same task twice → same result or no-op, (2) Isolation — concurrent tasks never interfere, (3) Monotonicity — main only accumulates, (4) Deterministic analysis — same spec+commit → same tier, (5) Non-regression — main passes tests before and after merge.

> **Practical v1 approach:** Specify invariants as runtime assertions checked every run. If violated → andon cord. Write property-based tests generating random task specs. Consider TLA+ spec as documentation (not model-checked initially).

---

8. Task Specification Schema

8.1 Input Grammar Design Patterns

Compilers define their input via formal grammars. Factory task specifications need equivalent rigor:

Grammar Formalisms

Formalism	Properties	Factory Applicability
BNF/EBNF	Declarative, unambiguous, tooling support	Task spec structure definition
PEG	Deterministic parsing, no ambiguity	Parsing freeform DEBT.md entries
JSON Schema	Industry standard, rich validation, tooling ecosystem	Task IR validation
OpenAPI	API contract specification	Inter-stage contracts
Cucumber/Gherkin	Human+machine readable Given/When/Then	Verification criteria specification

JSON Schema as Task IR Validator

JSON Schema provides the right balance of expressiveness and tooling for task IR validation: - **Type checking:** Ensures fields have correct types (string, number, array, enum) - **Required fields:** Prevents incomplete task specs from entering the pipeline - **Enum constraints:** Risk tier must be one of `["critical", "high", "medium", "low"]` - **Pattern validation:** File paths must match glob patterns - **Conditional validation:** If risk_tier is “critical”, then model must be “opus”

8.2 Existing Task Specification Models (Comparison)

System	Task Spec Model	Strengths	Weaknesses
SWE-bench	JSON: instance_id, repo, base_commit, problem_statement, FAIL_TO_PASS, PASS_TO_PASS	Binary verification, reproducible	No authorization constraints, no blast radius
DSPy	Python type annotations: input/output fields with semantic descriptions	Type-safe, composable, compilable	Prompt-focused, not pipeline-focused
Tekton	YAML: params (typed), results, workspaces, steps	Pipeline-native, Kubernetes-integrated	String-only results, no semantic typing
agents.json	OpenAPI-based: flows of API calls with links	Standard-based, discoverable	API-focused, not code-focused
Cucumber/Gherkin	Given/When/Then natural language	Human readable, testable	Not machine-optimizable, verbose
ABC ContractSpec	YAML: preconditions, invariants, governance, recovery	Formal, enforceable, hard/soft distinction	No task identity or execution planning

8.3 Minimum Viable Task Specification Fields

Synthesizing from all sources (SWE-bench + DSPy + Tekton + ABC), a Factory task spec requires:

Required Fields (Block if missing)

```yaml
# Identity (content-addressed)
task_id: string              # hash(source + base_commit)
source: string               # DEBT.md line reference or issue URL
description: string           # natural language intent
base_commit: string           # exact git state

# Scope (authorization boundary)
authorized_files:             # explicit allowlist
  - path: "src/auth/login.ts"
    operation: modify
  - path: "src/auth/__tests__/login.test.ts"
    operation: modify
risk_tier: critical | high | medium | low

# Verification (output contract)
postconditions:
  typecheck: "pnpm tsc --noEmit"
  tests_must_pass: ["src/auth/__tests__/login.test.ts"]
  forbidden_patterns: ["CLAUDE.md", "*.lock", ".env*"]
  max_files_changed: 5
```

Optional Fields (Populated by analysis passes)

```yaml
# Blast radius (computed by dependency analysis)
blast_radius:
  - path: "src/pages/login.tsx"    # imports from authorized files
    risk: medium
  - path: "src/api/auth.ts"
    risk: high

# Dependencies (computed by task graph analysis)
dependencies:
  - task_id: "abc123"
    relationship: blocks         # must complete first

# Execution plan (computed by model selection pass)
execution:
  model: "claude-sonnet-4-20250514"
  timeout_seconds: 300
  max_retries: 3
  constitution: ".factory/constitutions/high-risk.md"
  session_package:
    - ".factory/governance/risk-contract.md"
    - "tsconfig.json"
    - "src/auth/"                 # relevant source directory
```

8.4 Gherkin-Style Verification Criteria

For human-readable verification that's also machine-executable, adapt Gherkin:

```
Feature: Fix login form validation bug

  Background:
    Given the project typechecks cleanly
    And all existing tests pass
    And governance files are unchanged

  Scenario: Fix is applied correctly
    When the agent modifies src/auth/login.ts
    Then the login form validates email format
    And the test "login.test.ts::validates email" passes
    And no more than 3 files are changed
    And no files outside src/auth/ are modified

  Scenario: No regressions introduced
    When the agent's diff is applied
    Then all tests in src/ continue to pass
    And TypeScript compilation succeeds
    And no security vulnerabilities are introduced
```

> **Factory insight:** The Gherkin format serves a dual purpose: 1. **Human review:** Kieran can read the spec and verify it captures intent 2. **Machine execution:** Each Then clause maps to a verification command 3. **Agent context:** The Gherkin spec can be included in the agent's session package as a clear contract This bridges the gap between freeform DEBT.md descriptions and machine-readable postconditions.

---

9. Error Recovery Architecture

> This section synthesizes error recovery patterns from §2.10 (compiler error recovery), §4.4 (CI/CD retry), §5.4 (ABC recovery), §6.2 (saga compensation), and §10.1 (jidoka/andon).

9.1 Factory Error Taxonomy

Error Category	Examples	Recovery Strategy	Source
Transient	API rate limit, network timeout, model overloaded	Step retry with exponential backoff + jitter	Circuit breaker (§2.10)
Agent misbehavior	Modified governance files, committed to main, infinite loop	Immediate halt (andon cord), compensate, block task	ABC hard invariant (§5.4), Jidoka (§10.1)
Agent output quality	Typecheck fails, tests regress, security vulnerability	Stage retry (fresh worktree), up to N times	Erlang one_for_one (§2.10)
Merge conflict	Another task merged first, conflicts with this diff	Pipeline retry from agent execution with updated base	rest_for_one (§2.10)
Task specification error	Impossible task, missing files, circular deps	Mark blocked, escalate to human	Stall protocol (CLAUDE.md)
Infrastructure failure	Disk full, git corruption, process crash	Full pipeline retry with cleanup	one_for_all (§2.10)

9.2 Recovery Decision Tree

```
Error occurs at stage N
├── Is it a hard invariant violation?
│   ├── YES → ANDON CORD: halt immediately, compensate, block task, notify
│   └── NO → Continue
├── Is it a soft invariant violation?
│   ├── YES → Execute recovery action (within k steps)
│   │   ├── Recovery succeeds → Continue pipeline
│   │   └── Recovery fails → Treat as hard violation
│   └── NO → Continue
├── Is it a transient error?
│   ├── YES → Retry step (up to 3 times with backoff)
│   │   ├── Retry succeeds → Continue pipeline
│   │   └── All retries fail → Treat as stage failure
│   └── NO → Continue
├── Is it a stage failure?
│   ├── YES → Compensate stage N, retry from stage N (up to 2 times)
│   │   ├── Retry succeeds → Continue
│   │   └── All retries fail → Circuit breaker: mark task blocked
│   └── NO → Continue
└── Is it a pipeline-level failure?
    └── YES → Compensate all stages, full retry (once)
        ├── Retry succeeds → Continue
        └── Retry fails → Mark failed, escalate to human
```

9.3 Compensation Registry

Every transform stage MUST register a compensation action:

```yaml
        compensations:
          create_worktree:
            forward: "git worktree add .worktrees/{task_id} -b factory/{task_id}"
            compensate: "git worktree remove .worktrees/{task_id} && git branch -D factory/
{task_id}"

          agent_execution:
            forward: "claude --print --model {model} --session-package {pkg}"
            compensate: "kill {pid}; git checkout -- . (in worktree)"

          merge:
            forward: "git merge --ff-only factory/{task_id}"
            compensate: "git revert --no-edit HEAD"

          update_progress:
            forward: "append task_id to progress.md"
            compensate: "remove task_id from progress.md"
```

---

10. Manufacturing Patterns

10.1 Toyota Production System: Foundational Principles

The Toyota Production System (TPS) provides the manufacturing-tested foundations for Factory's quality model (Toyota Production System, Wikipedia TPS):

Jidoka (Automation with Human Touch)

Jidoka means **machines detect abnormalities and stop automatically**. Human intervention is only needed when the machine detects something it can't handle (Jidoka Article):

Jidoka Principle	Factory Application
Machine detects defect → stops	Verification stage detects failure → stops pipeline
Human investigates root cause	Developer reviews agent output on failure
Fix the process, not just the defect	Update task spec/constitution, not just re-run
Build quality in, don't inspect it out	Preconditions + invariants during execution, not just postconditions

> **Key insight:** Current Factory is "inspect quality out" — the agent runs freely, then we check if the output is valid. Jidoka says: **build quality constraints into the execution itself**. Use ABC hard invariants to stop the agent mid-execution when it violates a constraint, rather than waiting until after.

Andon Cord

Every worker can **stop the production line** when they find a defect (Toyota Andon, IT Revolution Andon):

- Pull the cord → team leader has ~60 seconds to respond
- If not resolved → entire line stops
- **Stopping is cheaper than letting defects flow downstream**
- Applied to software: Netflix Chaos Monkey, CI/CD build breaks

Factory andon implementation:

```
ANDON TRIGGERS (stop immediately):
- Agent modifies governance files (CLAUDE.md, risk contract)
- Agent commits to main branch (sandbox escape)
- Agent generates > 2× expected file count (runaway generation)
- Security review finds critical vulnerability
- TypeScript compilation fails after 3 retries

ANDON RESPONSE:
1. Kill agent process immediately
2. Notify developer (notification system)
3. Preserve all evidence (event log, diff, agent output)
4. Block task from re-entering pipeline until reviewed
```

Poka-Yoke (Mistake-Proofing)

Design the system so **mistakes are impossible** to make (Autonomation Wikipedia):

Poka-Yoke Type	Factory Application
Prevention: Make the wrong action physically impossible	Git worktree can't write to main branch — physically separated
Detection: Alert immediately when an error occurs	Pre-commit hook checks governance files are unchanged
Correction: Auto-fix minor errors	Auto-revert unauthorized file changes before verification

Factory poka-yoke checklist: - ✅ Git worktree (prevents direct writes to main) - ❌ No `.gitignore` in worktree to prevent governance file modifications - ❌ No file permission restrictions (agent can modify anything in worktree) - ❌ No pre-execution validation of task spec completeness - ❌ No auto-revert of unauthorized changes - **Missing:** Read-only mount of governance files in agent worktree

10.2 Theory of Constraints: Finding the Bottleneck

The Theory of Constraints (Goldratt) says **a system is only as fast as its slowest component** (Theory of Constraints, DZone ToC in Software):

The Five Focusing Steps

1. **Identify** the constraint (bottleneck)
2. **Exploit** the constraint (get maximum throughput from it)
3. **Subordinate** everything else to the constraint (don't overproduce upstream)
4. **Elevate** the constraint (increase capacity if needed)
5. **Repeat** (the constraint shifts)

Factory Bottleneck Analysis

Stage	Typical Duration	Is it the bottleneck?
Load harness	<1s	No
Baseline verification	10-30s	No
Risk classification	<1s	No
Model selection	<1s	No
Worktree creation	5-10s	No
Agent execution	**2-15 min**	**YES — always the bottleneck**
Verification	30-60s	No
Security review	1-3 min	Secondary bottleneck
Merge	<5s	No

> **Implications of agent execution as bottleneck:** 1. **Exploit:** Maximize agent success rate — better prompts, better context, better constitutions 2. **Subordinate:** Don't over-prepare tasks that the agent will fail. Use fast pre-checks to reject impossible tasks before they reach the agent. 3. **Elevate:** Run agents in parallel (Cursor 2.0 model). Reduce agent input size (smaller, focused session packages). 4. **Don't optimize non-bottlenecks:** Spending time optimizing the merge stage is waste. All optimization effort should go to agent execution quality and speed.

10.3 WIP Limits: Controlling Pipeline Concurrency

Work-in-Progress limits prevent overloading the system (WIP Limits, Little's Law and WIP):

Little's Law

```
Lead Time = WIP / Throughput
```

To reduce lead time: reduce WIP (don't start more work) or increase throughput (faster execution).

WIP Limits Applied to Factory

What to limit	Why	Suggested limit
Concurrent agent executions	Each consumes API tokens, CPU, memory	3-5 parallel agents
Tasks in verification queue	Verification is the secondary bottleneck	2× agent parallelism
Pending merge queue	Merges must be sequential (conflict risk)	1 merge at a time
Failed task retries	Prevent retry storms consuming all capacity	1 retry per task at a time

> **Siemens case study:** When Siemens Health Systems applied WIP limits, cycle time dropped from 71 to 43 days (42% improvement) — same teams, same capacity, just better flow control. Factory should learn from this: **don't start 20 tasks simultaneously**. Start 3-5, complete them, merge them, then start more.

10.4 Quality Gates vs Quality Built-In

Approach	How it works	Factory analogy
Quality gate (inspection)	Build first, inspect after	Current: agent runs freely, verify after
Quality built-in (jidoka)	Prevent defects during production	Future: preconditions, invariants, constrained execution

> **Factory should combine both:** 1. **Built-in:** Preconditions (baseline must pass), invariants (governance files read-only), constrained execution (authorized file set) 2. **Gates:** Postconditions (typecheck, tests), security review (separate agent), merge approval 3. **The insight:** Quality gates catch defects. Quality built-in prevents them. Both are needed, but built-in is cheaper and faster.

10.5 Statistical Process Control for Pipeline Health

Monitor pipeline health over time using manufacturing SPC principles:

Metric	What it measures	Control limits
Agent success rate	% of agent executions that pass verification	Target: >80%. Below 60% = process out of control
First-pass yield	% of tasks that succeed without retry	Target: >70%
Mean time to merge	Average time from task creation to merge	Track trend, alert on 2σ deviation
Defect escape rate	% of merged code that causes regressions	Target: <5%. Any increase = investigate
Retry rate per stage	How often each stage needs retry	Rising trend = deteriorating inputs or model

> **Factory should log these metrics via the event sourcing model (§6.1) and display a dashboard. If agent success rate drops below 60%, trigger an andon alert — something has changed in the codebase, model, or task quality.**

---

11. Existing Implementations Survey

11.1 OpenHands (formerly OpenDevin)

Architecture: Event stream + Docker sandbox + Action/Observation model (OpenHands Paper, OpenHands GitHub)

Aspect	Implementation	Factory Relevance
State management	Chronological event stream — all actions/observations recorded	✅ Factory should adopt: event sourcing for complete audit trail
Sandbox	Docker container per task session, workspace mounted	✅ Similar to git worktree but stronger (process isolation)
Agent loop	Action → Runtime executes → Observation → Agent decides next	✅ Matches Factory's execute → verify → decide loop
Tools	Bash shell, IPython, Chromium browser via BrowserGym	Partial — Factory agents only need shell + file editing
Verification	Integration tests (Appendix E), but not formalized	❌ No SWE-bench-style FAIL_TO_PASS/ PASS_TO_PASS
V1 (2025)	Modular SDK, opt-in sandboxing, reusable packages	✅ Shows trajectory toward composable, not monolithic

What they get right: Event stream as the single source of truth. Docker-level isolation. Clear action/observation boundary.

What they get wrong: No formal output contracts. No content-addressed caching. No pipeline structure (single agent loop, not multi-stage).

11.2 Aider

Architecture: Git-integrated pair programming with repo map (Aider, Aider Git Integration)

Aspect	Implementation	Factory Relevance
Git integration	Auto-commits LLM changes, dirty commits for uncommitted work	✅ Git as state store — proven approach
Repo map	Builds architecture map (file names, class/function signatures, imports) without sending full content	✅ Factory should build similar context for session packages
Commit attribution	"(aider)" appended to git author metadata	Nice-to-have: attribute Factory commits to task ID
Verification	`--git-commit-verify` runs pre-commit hooks (off by default with `--no-verify`)	❌ Verification is opt-in, not mandatory
Idempotency	None — re-running produces duplicate commits	❌ Same problem as current Factory
Pipeline	None — single interactive loop	❌ No multi-stage pipeline

What they get right: Git as the state management backbone. Repo map for efficient context selection.

What they get wrong: No idempotency. No pipeline structure. Verification is opt-in (and disabled by default!). No output contracts.

11.3 Devin (Cognition)

Architecture: Planner + Executor + Debugger loop with memory system (Devin, Cognition Blog)

Aspect	Implementation	Factory Relevance
Planning	Planner LLM creates step-by-step plan, self-critiques before execution	✅ Progressive refinement — matches MLIR lowering model
Execution	Lightweight executor selects tool (shell, editor, browser) per step	Similar to Factory stage execution
Debugging loop	Write → test → read logs → add debug statements → fix → retest	✅ Iterative refinement within agent execution
Sandbox	"Tightly sandboxed workspace"	✅ Confirms sandbox is industry standard
Memory	Vectorized code snapshots + full replay timeline of commands, diffs, browser tabs	✅ Event sourcing + vector store for context
Adaptive planning	Plan evolves as new information is discovered	Matches exploratory research mandate
Verification	Iterative until tests pass, but human review for final acceptance	Semi-formal — tests but no formal contracts

What they get right: Adaptive planning (plan changes during execution). Iterative debug loop. Full replay timeline (event sourcing). Sandbox isolation.

What they get wrong: No formal output contracts. No content-addressed caching. No idempotency guarantee. Human-in-the-loop verification (doesn't scale for Factory's autonomous goal).

11.4 Cursor 2.0 (Composer)

Architecture: Multi-agent, parallel execution with isolated workspaces (Cursor 2.0, Cursor Architecture Guide)

Aspect	Implementation	Factory Relevance
Parallel execution	Up to 8 agents simultaneously, each in isolated codebase copy	✅ Factory parallel model: git worktrees per task
Workspace isolation	Git worktrees or remote worker sandboxes	✅ Validates Factory's worktree approach
Conflict prevention	Sandboxing prevents file conflicts between agents	✅ Factory merge queue needed for sequential merge
Model	Custom Composer (MoE, RL-trained on real codebases with tools)	Interesting — RL on tool use improves agent quality
Multi-file diffs	Aggregated diffs reviewable in one place	✅ Factory should present unified diff for review
Training	Many rollouts in parallel, score outputs, keep best strategies	DSPy-like optimization of tool-calling patterns

What they get right: Parallel execution with isolation. Worktree-based sandboxing at scale (8 agents!). RL-trained tool use.

What they get wrong: No pipeline structure (agent-first, not pipeline-first). No formal verification beyond user review. No idempotency. No event sourcing.

11.5 Cross-Implementation Pattern Analysis

What ALL implementations get right

1. **Sandbox isolation** — every system uses some form of isolated execution environment
2. **Git integration** — git is the universal state backbone
3. **Iterative execution** — agents loop (plan → act → observe → refine)
4. **Context management** — all build some form of repo map / context window

What ALL implementations get wrong

1. **No idempotency** — re-running produces different/duplicate results

2. **No formal output contracts** — outputs are validated heuristically, not formally
3. **No content-addressed caching** — identical tasks are re-executed from scratch
4. **No pipeline structure** — all are agent-first, not pipeline-first
5. **No compensating transactions** — failure leaves orphaned state
6. **No event sourcing** — OpenHands is the exception, but even they don't use it for recovery

What Factory can uniquely provide

1. **Pipeline-first architecture** — compiler-inspired multi-stage pipeline with typed IR
2. **Content-addressed caching** — Bazel/Nix model applied to agent output
3. **Formal output contracts** — SWE-bench + ABC model for verification
4. **Idempotency by design** — task hash → cache lookup → skip if cached
5. **Compensating transactions** — saga pattern for clean failure recovery
6. **Progressive lowering** — MLIR model: raw spec → structured spec → execution plan → verified code → merged commit

> **This is Factory's unique value proposition.** Existing tools focus on making agents smarter. Factory focuses on making the pipeline around the agent correct, idempotent, and verifiable. These are orthogonal — Factory can use any agent (Claude, GPT, open-source) as its "backend" while providing correctness guarantees that no current system offers.

---

12. SYNTHESIS: Factory Compiler Architecture Spec

12.1 Architecture Overview

Factory is a **compiler-like pipeline** that takes task specifications as input and produces verified, merged code as output. Drawing from all 10 research angles, the architecture combines:

- **MLIR progressive lowering** (Angle 1) for multi-level task refinement
- **Bazel/Nix content-addressing** (Angle 2) for idempotency and caching
- **Dagger immutable DAG** (Angle 3) for pipeline execution model
- **ABC formal contracts** (Angle 4) for agent behavioral guarantees
- **Event sourcing + sagas** (Angle 5) for state management and recovery
- **TLA+ invariants** (Angle 6) as runtime assertions
- **JSON Schema + Gherkin** (Angle 7) for typed task specs
- **Erlang supervision + circuit breakers** (Angle 8) for error recovery
- **Toyota jidoka + andon** (Angle 9) for quality built-in
- **OpenHands event streams + Cursor parallel execution** (Angle 10) for proven patterns

12.2 The Factory Pipeline (Redesigned)

```
FRONTEND (Task Specification → Structured IR)
├── Stage F1: Parse Task        [Analysis] Parse DEBT.md item → raw task object
├── Stage F2: Validate Spec     [Analysis] JSON Schema validation → reject invalid specs
├── Stage F3: Dedup Check       [Analysis] Hash(source + base_commit) → check action cache
│   └── CACHE HIT → Skip to merge (return cached verified diff)
└── Stage F4: Risk Classification [Analysis] File paths → risk tier + blast radius

MIDDLE-END (Structured IR → Execution Plan)
├── Stage M1: Dependency Analysis [Analysis] Import graph → dependency set
├── Stage M2: Conflict Prediction [Analysis] git merge-tree vs in-flight tasks
├── Stage M3: Model Selection     [Analysis] Risk tier → model + timeout + retries
├── Stage M4: Constitution Gen    [Transform] Materialize CLAUDE.md + blast radius +
contracts
└── Stage M5: Session Package     [Transform] Build context package for agent

BACKEND (Execution Plan → Verified Code)
├── Stage B1: Create Worktree    [Transform] git worktree add, install deps
├── Stage B2: Execute Agent      [Transform] Claude CLI in worktree (IMPURE — fixed-output)
│   └── ABC Contract Enforcement: preconditions, hard/soft invariants, governance checks
├── Stage B3: Verify Output      [Analysis] Typecheck + tests (FAIL_TO_PASS + PASS_TO_PASS)
├── Stage B4: Security Review    [Analysis] Separate agent reviews diff for vulnerabilities
└── Stage B5: Merge              [Transform] git merge --ff-only, write evidence

LINKER (Post-Merge)
└── Stage L1: Update State       [Transform] Update progress, invalidate caches, emit
events
```

12.3 Key Architectural Decisions

Decision	Rationale	Source
Separate analysis from transform stages	Analysis stages are cacheable and idempotent. Transform stages need compensation.	LLVM Pass Manager (§2.4)
Content-addressed action cache	`hash(task_spec + base_commit + model_version + constitution)` → cached diff. Solves idempotency.	Bazel AC/CAS (§3.3)
Progressive lowering through 4 levels	Raw task → structured spec → execution plan → verified code. Each level adds constraints.	MLIR dialects (§2.5), GCC GENERIC→GIMPLE→RTL (§2.7)
Event sourcing for state	Every stage transition is an immutable event. Enables crash recovery, audit, metrics.	Event sourcing (§6.1)
Saga orchestration with compensation registry	Every transform stage has a registered undo action. Central orchestrator manages.	Saga pattern (§6.2)
ABC contracts per stage	Pre/post/invariant conditions specified in YAML. Hard invariants → immediate halt.	ABC (§5.4), DbC (§7.2)
Fixed-output derivation for agent execution	Agent is impure (non-deterministic). Treat like Nix: let it run in sandbox, verify output against declared postconditions.	Nix (§3.2)
rest_for_one supervision	On failure, restart from failed stage (not beginning). Preserve cached analysis results.	Erlang OTP (§2.10)
Circuit breaker at pipeline level	3 failures → mark blocked, escalate to human. Prevent retry storms.	Circuit breaker (§2.10)
Andon cord for hard invariant violations	Governance file modification, sandbox escape → immediate halt + notify + preserve evidence.	Toyota jidoka (§10.1)
		SWE-bench (§5.2)

Decision	Rationale	Source
SWE-bench verification model	FAIL_TO_PASS + PASS_TO_PASS for every task. Binary pass/fail.	
Git as primary state store	Task manifests as JSON, events as files, worktrees for isolation, commits for CAS.	Git concurrency (§6.3)
WIP limits on concurrent agents	3-5 parallel agents max. 1 merge at a time. Little's Law: reduce WIP to reduce lead time.	Theory of Constraints (§10.2)

12.4 Task IR Schema (Final)

```yaml
# Level 1: Identity (populated by F1-F2)
task_id: string              # hash(source + base_commit)
source: string               # DEBT.md reference or issue URL
description: string          # natural language intent
base_commit: string          # exact git state
idempotency_key: string      # hash for action cache lookup

# Level 2: Scope (populated by F4, M1)
authorized_files:
  - { path: string, operation: create|modify|delete }
blast_radius:
  - { path: string, risk: high|medium|low }
dependencies:
  - { task_id: string, relationship: blocks|informs }
risk_tier: critical|high|medium|low

# Level 3: Contract (populated by F2, ABC)
preconditions:
  - { type: typecheck|test|lint, command: string, expected: pass }
postconditions:
  - { type: test, tests_must_pass: string[], tests_must_fix: string[] }
  - { type: typecheck, command: string }
  - { type: diff_constraint, max_files: number, forbidden: string[] }
invariants:
  hard: ["governance_files_unchanged", "no_main_commits", "authorized_files_only"]
  soft: ["timeout_warning_at_80pct", "file_count_warning_at_2x"]
recovery:
  - { trigger: "soft_file_violation", action: "revert_unauthorized_changes" }

# Level 4: Execution (populated by M3-M5, B1)
model: string
worktree_path: string
session_package: string[]
constitution_path: string
timeout_seconds: number
max_retries: number
```

12.5 Pass Properties and Scheduling

Pass Property Flags (GCC-style)

Each pass declares what properties the IR must have (require) and what it produces (provide):

Property Flag	Set by	Required by
`PROP_parsed`	F1: Parse Task	F2-F4
`PROP_validated`	F2: Validate Spec	F4, M1-M5
`PROP_risk_classified`	F4: Risk Classification	M3: Model Selection
`PROP_dependencies_analyzed`	M1: Dependency Analysis	M2: Conflict Prediction
`PROP_planned`	M3-M5 (all complete)	B1: Create Worktree
`PROP_baseline_ok`	(implicit: preconditions pass)	B2: Execute Agent
`PROP_executed`	B2: Execute Agent	B3-B4: Verification
`PROP_verified`	B3-B4 (all pass)	B5: Merge

LLVM rule enforced: Transform passes can only REQUIRE analysis passes, never other transforms. This prevents cycles and makes scheduling deterministic.

Caching Architecture (Bazel/Nix Model)

Action Cache:

```
ActionCache[hash(task_spec + base_commit + model_version + constitution_hash)]
  → { output_diff_hash, commit_sha, evidence_path }
```

- Cache hit at F3 → skip entire pipeline, return cached result
- Cache miss → full execution, write result to cache at L1
- Git IS the CAS — objects already stored by SHA

Analysis Cache (LLVM AnalysisManager model):

```
{
  "import_graph": { "input_hash": "sha256(...)", "value": {...}, "valid": true },
  "risk_classification": { "input_hash": "sha256(...)", "value": "medium", "valid": true }
}
```

- Shared across tasks targeting the same base_commit
- Invalidated when a merge changes relevant files
- **Early cutoff (Shake):** If re-analysis produces same result → skip downstream

Execution Model

Scheduler: Suspending (from Build Systems à la Carte): 1. Parse all DEBT.md items → L0 Task IRs 2. Check action cache → filter out cached 3. Run analysis (parallelizable across tasks) 4. Build dependency graph 5. Schedule with WIP limit (3-5 concurrent agents) 6. Suspend tasks waiting for dependencies 7. On completion → update caches, resume suspended

Agent Execution: Fixed-Output Derivation (from Nix): The agent is the ONE impure step. Handle like Nix handles network: 1. Let agent run in sandbox (impure, non-deterministic) 2. Verify output against postconditions (SWE-bench model) 3. Verification fails → "build" fails → retry or escalate 4. Verification passes → output is cacheable

Error Recovery Supervision Tree (Erlang OTP)

```
Pipeline Supervisor [circuit breaker: max 3 failures per task]
├── Frontend [one_for_one: restart failed pass only]
│   ├── F1: TaskParser
│   ├── F2: SchemaValidator
│   └── F3: DedupCheck
├── Analysis [one_for_one: restart failed analysis]
│   ├── F4: RiskClassifier
│   ├── M1: DependencyAnalyzer
│   └── M2: ConflictPredictor
├── Execution [rest_for_one: restart from failure onward]
│   ├── M3-M5: Planning
│   ├── B1: WorktreeProvisioner
│   ├── B2: AgentExecutor ← most likely failure
│   └── B3-B4: Verification
└── Linker [one_for_one]
    ├── B5: MergeExecutor
    └── L1: StateUpdater
```

Retry policies per group (Argo model):

Group	Policy	Max	Backoff	Rationale
Frontend	OnError	2	None	Infrastructure failures only
Analysis	OnError	2	None	Infrastructure failures only
Agent Execution	Always	3	30s→60s→120s	Non-deterministic, retry often helps
Verification	Never (escalate to re-execute)	0	—	Deterministic: same input → same failure
Merge	OnError	5	1s exponential	Git lock contention is transient

Saga Compensation Registry:

Transform	Compensating Transaction
B1: Create Worktree	`git worktree remove --force`
M4: Constitution Gen	Delete materialized file
B2: Agent Execute	`git checkout -- .` in worktree
B5: Merge	`git revert <commit>` on main
L1: Update State	Remove state entries

Concurrency Model

```
┌──────────────────────────────────┐
│        TASK SCHEDULER            │
│  WIP Limit: 5 agents             │
│  Merge Limit: 1 at a time        │
└──────────┬──────┬──────┬─────────┘
           │      │      │
 ┌─────────▼─┐ ┌─▼─────┐ ┌▼──────┐
 │ WT-1  │ │WT-2 │ │WT-3  │   (git worktrees)
 │Task A │ │TaskB│ │TaskC │
 └───────┘ └─────┘ └───────┘
     │        │       │
 ┌───▼────────▼───────▼──┐
 │     MERGE QUEUE       │
 │ Sequential, FIFO      │
 │ Conflict check before │
 │ each merge            │
 └───────────────────────┘
```

- Conflict check: `git merge-tree` before actual merge
- If files overlap with recently merged task → rebase + re-verify

Migration from orchestrate.sh

Priority	Change	Impact
1	Task IR as JSON (replace bash vars)	Enables everything else
2	Event logging (JSONL)	Observability, debugging
3	Action cache (hash-based dedup)	Solves idempotency (#1 bug)
4	Compensating transactions	Clean failure recovery
5	SWE-bench verification	Output quality guarantee
6	Diff constraint checking	Scope enforcement
7	Andon cord + notifications	Failure visibility
8	Analysis caching	Performance
9	Parallel execution + merge queue	Throughput
10	Container-based sandboxing	Security (v2)

12.6 Deep Dive: MLIR Dialect Conversion as Factory Model

The MLIR dialect conversion framework (MLIR Dialect Conversion) provides a precise engineering model for Factory's progressive lowering:

Three conversion modes map to Factory execution strategies: - **Partial conversion** = progressive lowering: convert what's ready, leave rest unchanged. Factory stages populate IR levels incrementally — L1 identity first, then L2 scope, then L3 contracts. - **Full conversion** = final merge gate: all IR levels must be populated before backend execution. If any field is missing, conversion fails. - **Analysis conversion** = dry-run validation: check if the pipeline *could* succeed without executing transforms. Factory pre-flight check.

Transitive lowering eliminates redundant pass specifications. If risk_classification depends on file_paths and model_selection depends on risk_tier, the framework automatically chains them. Factory doesn't need explicit cross-stage wiring — the IR schema serves as the implicit contract.

Rollback mode vs no-rollback mode maps directly to Factory's compensation question. Rollback mode delays IR modifications, enabling pattern backtracking on failure. This is exactly the saga compensation model: delay side effects, commit only on success, roll back on failure. Factory should default to rollback mode (saga compensation) for transforms and no-rollback mode for analyses (read-only, no undo needed).

ConversionTarget legality checking validates the IR *before* attempting conversion. Factory should validate task IR schema completeness before each stage — not after. This catches missing fields, invalid types, and constraint violations before wasting agent execution time.

12.7 Deep Dive: Nix Store Path Computation and Content-Addressing

Nix's store path model (Nix Store Paths, CA Derivations) clarifies Factory's caching design:

Input-addressed vs content-addressed derivations: - Input-addressed: `hash(derivation contents)` — pessimistic. Any upstream change → new path, even if output identical. Current Factory approach: `hash(task_spec + commit + model)`. - Content-addressed: `hash(output bytes)` — optimistic. Same output → same path regardless of inputs. Enables early cutoff: if re-running task produces identical diff, downstream steps don't re-execute.

Factory should use BOTH: 1. Input-addressed for action cache lookup (fast, deterministic, no execution needed) 2. Content-addressed for cross-task dedup (two different task specs that produce identical diffs share the same cached result) 3. Fixed-output derivations for agent execution (declared postconditions, sandbox escape for impure execution, output verified against contract)

The "modulo FOD" optimization prevents rebuild cascades: when an FOD's derivation path changes but its output is the same, Nix substitutes the output hash instead of the derivation hash in downstream computations. Factory analog: if the constitution template changes but produces the same materialized constitution for a given task, downstream stages don't re-run.

12.8 Deep Dive: LangGraph vs Factory Architecture

LangGraph 1.0 (LangGraph) is the production-ready agent orchestration framework most often compared with pipeline architectures:

Architecture comparison:

Dimension	LangGraph	Factory
Core model	Agent graph (nodes = agents)	Compiler pipeline (stages = passes)
State	Centralized TypedDict + reducers	Progressive IR + event log
Checkpointing	After every node (Postgres/SQLite)	Event sourcing (JSONL)
Idempotency	None (re-run = new execution)	Content-addressed action cache
Concurrency	Multi-agent state conflicts	Git worktree isolation
Recovery	Time-travel replay	Saga compensation
Human-in-loop	Interrupt at any node	Andon cord on invariant violation
Compilation	Graph validated before execution	IR schema validated per stage

Key LangGraph insight: >75% of multi-agent systems become unmanageable past 5 agents due to state coordination complexity. Factory avoids this because agents don't coordinate — each runs in an isolated worktree with a fixed contract. The pipeline coordinates, not the agents.

What Factory should adopt from LangGraph: Graph compilation (validate pipeline structure before execution) and time-travel debugging (replay from any event to reproduce failures).

12.9 Deep Dive: Clash and Merge Conflict Prediction

Clash (clash-sh/clash) solves a critical Factory gap: predicting merge conflicts between concurrent agent worktrees *before* they happen.

How it works: 1. Discover all active worktrees 2. For each pair, find merge base 3. Run `git merge-tree` (in-memory three-way merge via gix library) 4. Report conflicts without modifying repository

Factory integration points: - **Pre-scheduling:** Before dispatching parallel agents, Clash-check all worktree pairs. Conflicts → serialize. No conflicts → parallelize. - **Pre-merge:** After agent completion, merge-tree against current main HEAD to catch drift from other merged tasks. - **Watch mode:**

Continuous monitoring during agent execution to detect emerging conflicts as agents write files. - **Claude Code hooks:** Clash integrates with Claude Code hooks to check files before writes — directly applicable to Factory's agent execution stage.

12.10 Deep Dive: Property-Based Testing for Pipeline Correctness

Hypothesis stateful testing (Hypothesis Stateful) provides the testing model for Factory pipeline verification:

RuleBasedStateMachine for Factory:

```
class FactoryPipelineTest(RuleBasedStateMachine):
    tasks = Bundle("tasks")
    worktrees = Bundle("worktrees")

    @rule(target=tasks, spec=task_specs())
    def parse_task(self, spec): ...

    @rule(task=tasks, target=worktrees)
    def create_worktree(self, task): ...

    @invariant()
    def governance_unchanged(self):
        assert governance_hash() == INITIAL_HASH

    @invariant()
    def no_duplicate_tasks(self):
        assert len(set(self.active_tasks)) == len(self.active_tasks)
```

Hypothesis generates random sequences of pipeline operations + failure injections, checking invariants after every step. This catches: concurrent merge races, compensation gaps, state corruption from partial failures, ordering violations. **No existing agent pipeline uses property-based testing for pipeline correctness** — this is a novel application.

12.11 Deep Dive: Container vs Worktree Isolation — Factory's Largest Security Gap

Git worktrees provide code isolation only — separate working directory, index, and HEAD. But the agent shares the host's filesystem, network, ports, Docker daemon, and all other system resources. This is NOT a security boundary (Container Use - InfoQ, Beyond Worktrees).

Isolation hierarchy (weakest → strongest):

Level	Filesystem	Network	Resources	Host Access	Used By
Git worktree	Separate workdir	Shared	Shared	Full	Factory (current)
Docker container	Private rootfs	Namespace	cgroups	None	OpenHands, SWE-agent
MicroVM	Full VM	Full isolation	Dedicated	None	Codex, Docker Sandboxes

The hybrid approach (Codex model): Container for security + worktree for git isolation *within* the container. This gives both security (no host access) and git ergonomics (worktree operations, branch management).

Factory migration path: - v1: Worktree + read-only governance mounts + restricted permissions (remove `bypassPermissions`) - v2: Docker container per agent with worktree inside, resource limits, network isolation - v3: MicroVM for critical-tier tasks

> ⚠️ **CRITICAL:** Factory currently runs agents with `bypassPermissions` in a worktree with full host access. This is the largest security gap in the current architecture. Any agent can read/write any file on the host, make network requests, and access credentials.

12.12 Deep Dive: Merge Queue Architecture

Factory needs a merge queue to serialize concurrent agent outputs. The industry has converged on a clear pattern (Mergify, bors-ng):

Speculative execution (Mergify model): Test multiple PRs in parallel, each rebased atop the expected post-merge state of all prior queue entries. If task N fails, re-verify N+1, N+2 with updated base (excluding N). This maximizes throughput while maintaining serialized merge ordering.

Factory merge queue design: 1. Agent completes → diff enters merge queue (FIFO by completion time) 2. Pre-merge check: `git merge-tree` against expected post-merge main HEAD 3. If clean: merge with CAS (compare-and-swap — abort if main has advanced since check) 4. If conflict: rebase on actual main → re-verify → re-enqueue 5. Speculative: next 3 tasks verify in parallel against expected future states

12.13 Deep Dive: Cross-Run Learning (v2 Architecture)

Factory v1 produces an event sourcing log rich enough to power v2's learning loop:

Data captured per task: `{task_type, constitution_hash, model, authorized_files, outcome, duration, retry_count, verification_scores, failure_mode}`

Learning applications: 1. **DSPy constitution optimization:** Treat constitution template as a DSPy program. Bootstrap few-shot examples from successful completions. Optimize instruction wording per task type via MIPROv2 Bayesian search. 2. **Success rate tracking:** SPC p-chart per task type. When rate drops below LCL → investigate model degradation, context quality, or task difficulty shift. 3. **Failure pattern analysis:** Cluster failures by root cause (context overflow, wrong file scope, test environment issues). Update constitution templates to address systematic patterns. 4. **Model selection calibration:** Track success rate × cost per model per risk tier. Adjust selection heuristics based on empirical data.

> ⚠️ **This is v2+ work.** Requires v1 baseline data before optimization is meaningful.

12.14 Deep Dive: Agent Performance Reality Check

SWE-bench Verified top performers (March 2026) (SWE-bench): - Claude Opus 4.5: ~80.9% - Claude Opus 4.6 (Thinking): ~79.2% - GPT 5.4: ~77.2% - mini-swe-agent (100 lines): >74%

Implication for Factory: Even the best agents fail ~20% of the time. With 3 retries at 80% success: `1 - (0.2)^3 = 99.2%` eventual success rate. This confirms: 1. **Retry budget is essential** — single-shot execution wastes agent capacity 2. **Failure is a first-class outcome** — pipeline must handle it gracefully (compensation, escalation) 3. **Verification is non-negotiable** — without FAIL_TO_PASS/PASS_TO_PASS, you can't distinguish the 80% from the 20% 4. **Pipeline correctness > agent quality** — a correct pipeline with a 74% agent (mini-swe-agent) outperforms a broken pipeline with an 80% agent

12.15 What Factory Gets That Nobody Else Has

1. **Idempotency by construction** — content-addressed task hashing + action cache
2. **Typed stage contracts** — every stage has pre/post/invariant conditions
3. **Progressive lowering** — 4-level IR refinement, not one-shot agent dispatch
4. **Formal compensation** — saga pattern ensures clean recovery on any failure
5. **Pipeline-first, not agent-first** — the pipeline is correct regardless of which agent backend is used
6. **Quality built-in** — ABC hard invariants during execution, not just postcondition inspection
7. **Verifiable outputs** — SWE-bench FAIL_TO_PASS/PASS_TO_PASS model for every task
8. **Cached analysis** — expensive computations (import graph, risk analysis) computed once, invalidated on change

9. **Observable** — event sourcing provides complete audit trail and pipeline health metrics
10. **Concurrent-safe** — git worktrees + merge-tree conflict prediction + WIP limits

12.7 Known Gaps and Future Work

1. **Agent quality optimization:** Pipeline verifies output but can't make the agent produce better code. DSPy-style prompt compilation (§5.1) for v2.
2. **Semantic correctness:** Tests catch regressions, but "does this fix match user intent?" requires human review or more sophisticated evaluation.
3. **Cross-task semantic conflicts:** Merge queue handles file-level conflicts, but two tasks that independently change related behavior need dependency-aware scheduling.
4. **Dynamic task decomposition:** Agent discovers subtasks mid-execution — needs a mechanism to spawn child tasks.
5. **Learning from failures:** No automatic feedback loop from failed runs to task spec / constitution improvement.
6. **Model cost optimization:** Risk-tier-based model selection is a heuristic; need empirical calibration.

---

13. Sources

- Compilers: Principles, Techniques, and Tools (Dragon Book) — Aho, Lam, Sethi, Ullman
- Intermediate Code Generation in Compiler Design - GeeksforGeeks
- Grouping of Phases in Compiler Design - GeeksforGeeks
- Single Pass vs Multi-Pass Compilers - GeeksforGeeks
- Understanding LLVM Passes and Their Types - CompilerSutra
- LLVM New Pass Manager Documentation
- SSA Form - Wikipedia
- Compiler - Wikipedia
- Intermediate Representation - Wikipedia

- The Complete Evolution of LLVM Pass Infrastructure - CompilerSutra
- Writing an LLVM Pass (Legacy PM) - LLVM Docs
- MLIR Introduction - Stephen Diehl
- MLIR: A Compiler Infrastructure for the End of Moore's Law (Paper)
- MLIR Official Site
- GCC Compilation Pipeline - DeepWiki
- GCC Internals Overview - gcc-python-plugin docs
- GIMPLE - GCC Internals
- Tree-SSA Passes - GCC Internals
- Symbol Table in Compiler - GeeksforGeeks
- LLVM AnalysisManager Class Reference
- Persistent Data Structure - Wikipedia
- SWE-bench Datasets Guide
- DSPy Signatures
- agents.json Specification
- Tekton Pipeline API
- Error Recovery Strategies in Compiler Design - GeeksforGeeks
- Erlang Supervisor Behaviour
- Erlang OTP Supervision Trees - Adopting Erlang
- Microservices Resilience Patterns - GeeksforGeeks
- Build Systems à la Carte: Theory and Practice (Mokhov, Mitchell, Peyton Jones)
- Nix Hermeticity - Zero to Nix
- Nix Content-Addressing Derivation Outputs
- Bazel's Remote Caching Explained - BuildBuddy
- Bazel Remote Caching Docs
- Shake Build System
- Tekton Pipelines Docs
- Argo Workflows
- Dagger Architecture Overview - DeepWiki
- Dagger Built-In Caching
- AWS CodePipeline Stage Retry

- Harness Failure Strategies
- The Andon Cord - IT Revolution
- DevOps and the Andon Cord
- DSPy Framework
- DSPy MIPROv2 Optimizer
- SWE-bench Evaluation Guide
- SWE-bench GitHub
- METR - Measuring Autonomous AI Capabilities
- Agent Behavioral Contracts (ABC)
- NVIDIA: Sandboxing Agentic Workflows
- SkillFortify: Formal Analysis for Agent Skills
- OpenAI Codex Sandboxing
- AutoGen Multi-Agent Framework
- AutoGen Termination Conditions
- TLA+ - Leslie Lamport
- TLA+ - Wikipedia
- Design by Contract - Wikipedia
- Design by Contract - Meyer
- Property-Based Testing for System Invariants
- Saga Pattern - Microservices.io
- Temporal: Mastering Saga Patterns
- Event-Driven Architecture Patterns - Solace
- CRDTs - CRDT.tech
- CRDT - Wikipedia
- Git Worktrees for Concurrent Development - Ken Muse
- Git Concurrency in GitHub Desktop
- Toyota Production System - Wikipedia
- Toyota Production System - Toyota Global
- Toyota Andon Cord
- Jidoka in TPS - OrcaLean
- Autonomation - Wikipedia
- Theory of Constraints - Wikipedia

- Theory of Constraints in Software - DZone
- WIP Limits - Agility at Scale
- Little's Law and WIP Limits
- OpenHands Paper - ICLR 2025
- OpenHands GitHub
- Aider Git Integration
- Aider Git Integration - DeepWiki
- Devin Agents 101
- Cognition: Rebuilding Devin for Claude Sonnet
- Cursor 2.0 Release
- Cursor 2.0 Agent Architecture Guide
- METR Task Standard
- METR Blog: Task Standard
- AISI Inspect Sandboxing
- Argo Workflows Retries
- Tekton Tasks Documentation
- Tekton Workspaces
- Dagger Caching
- DSPy Paper - arXiv
- MLIR Dialect Conversion
- MLIR Toy Tutorial Ch5 - Partial Lowering
- MLIR Toy Tutorial Ch6 - Lowering to LLVM
- What's in a Nix Store Path - Farid Zakaria
- CA-Derivations - NixOS Wiki
- Derivation Outputs in a Content-Addressed World - Tweag
- Reflecting on the Shake Build System - Neil Mitchell
- LangGraph Official
- LangGraph Architecture Analysis 2025
- Clash - Merge Conflict Detection for Worktrees
- git merge-tree Documentation
- Hypothesis Stateful Testing
- Property-Based Testing 101 (2026)

- CrewAI Tasks
- CrewAI Hierarchical Process
- WAL Foundation for Reliability
- Write-Ahead Logging - Wikipedia
- Alloy 6 - Official
- Alloy 6: It’s About Time - Hillel Wayne
- METR Task Standard - GitHub
- METR Task Development Guide
- Aider Repository Mapping - DeepWiki
- Aider Repository Map Docs
- Optimistic Concurrency Control - Wikipedia
- Trunk Optimistic Merging
- Martin Fowler: Given-When-Then
- JSON Schema Conditionals
- Erlang Supervisor Behaviour (v28)
- Learn You Some Erlang: Supervisors
- SPC - ASQ
- Poka-Yoke in Software
- Kanban WIP Limits - Businessmap
- OpenHands V1 SDK Paper
- OpenHands V1 Blog
- GitHub Copilot Workspace
- Replit Agent 3 - InfoQ
- Replit Automated Self-Testing
- Amazon Q Developer
- SWE-agent Architecture
- mini-swe-agent
- Dagger Function Caching
- SWE-bench Leaderboard
- SWE-rebench
- Bazel Remote Execution API
- Container Use - InfoQ

- Beyond Worktrees: Isolation for Agents
- Mergify Merge Queue
- bors-ng
- Agent Memory Survey (2026)
- Multi-Agent Memory Engineering - MongoDB